

82. Physical Coding Sublayer (PCS) for 64B/66B, type 40GBASE-R and 100GBASE-R

82.1 Overview

82.1.1 Scope

This clause specifies the Physical Coding Sublayer (PCS) that is common to two families of (40 Gb/s and 100 Gb/s) Physical Layer implementations, known as 40GBASE-R and 100GBASE-R. The 40GBASE-R PCS is a sublayer of the 40 Gb/s PHYs listed in Table 80–1. The 100GBASE-R PCS is a sublayer of the 100 Gb/s PHYs listed in Table 80–1. The terms 40GBASE-R and 100GBASE-R are used when referring generally to Physical Layers using the PCS defined in this clause.

Both 40GBASE-R and 100GBASE-R are based on a 64B/66B code. The 64B/66B code supports transmission of data and control characters, while maintaining robust error detection. Data distribution is introduced to support multiple lanes in the Physical Layer. Part of the distribution includes the periodic insertion of an alignment marker, which allows the receive PCS to align data from multiple lanes.

82.1.2 Relationship of 40GBASE-R and 100GBASE-R to other standards

Figure 82–1 depicts the relationships among the 40GBASE-R and 100GBASE-R sublayers (shown shaded), the Ethernet MAC and Reconciliation Sublayers, and the higher layers.

This clause borrows heavily from Clause 49. 64B/66B encoding is reused with appropriate changes made to support 8 byte alignment versus 4 byte alignment in Clause 49. In addition to 64B/66B encoding, there is a methodology to add alignment markers and distribute data to multiple lanes.

82.1.3 Summary of 40GBASE-R and 100GBASE-R sublayers

Figure 82–1 shows the relationship of the 40GBASE-R PCS and 100GBASE-R PCS sublayers (shown shaded) with other sublayers to the ISO Open System Interconnection (OSI) reference model.

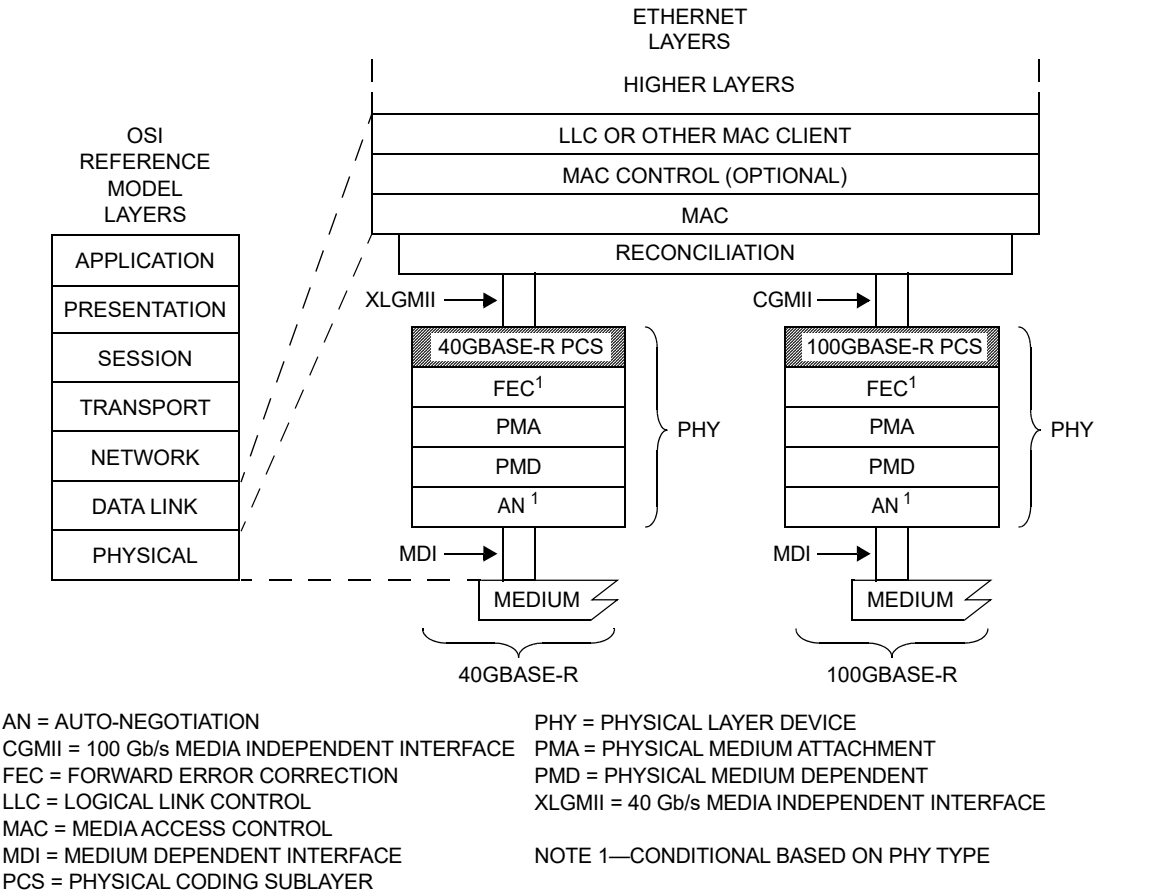


Figure 82–1—40GBASE-R and 100GBASE-R PCS relationship to the ISO/IEC Open Systems Interconnection (OSI) reference model and IEEE 802.3 Ethernet model

82.1.3.1 Physical Coding Sublayer (PCS)

The PCS service interface is the Media Independent Interface (XLGMII/CGMII), which is defined in Clause 81. The 40 Gb/s variant of this is called the 40 Gb/s Media Independent Interface (XLGMII) and the 100 Gb/s variant of this interface is called the 100 Gb/s Media Independent Interface (CGMII). The XLGMII/CGMII provides a uniform interface to the Reconciliation Sublayer for all 40 Gb/s and 100 Gb/s PHY implementations.

The 40GBASE-R and 100GBASE-R PCSs provide all services required by the XLGMII/CGMII, including the following:

- a) Encoding (decoding) of eight XLGMII/CGMII data octets to (from) 66-bit blocks (64B/66B).
- b) Transferring encoded data to (from) the PMA.
- c) Compensation for any rate differences caused by the insertion or deletion of alignment markers or due to any rate difference between the XLGMII/CGMII and PMA through the insertion or deletion of idle control characters.
- d) Determining when a functional link has been established and informing the management entity via the MDIO when the PHY is ready for use.

82.1.4 Inter-sublayer interfaces

The upper interface of the PCS may connect to the Reconciliation Sublayer through the XLGMII/CGMII. The lower interface of the PCS connects to the PMA sublayer to support a PMD. If the optional BASE-R FEC sublayer is implemented (see Clause 74) and an optional physical instantiation, i.e., XLAUI or CAUI-n, is not implemented directly below the PCS sublayer, then the lower interface connects to the BASE-R FEC sublayer. For Physical Layers that use Clause 91 RS-FEC, if an optional physical instantiation (i.e., CAUI-n) is not implemented directly below the PCS sublayer, then the lower interface connects to the RS-FEC sublayer. The 40GBASE-R PCS has a nominal rate at the PMA/FEC service interface of 10.3125 Gtransfers/s per PCS lane, which provides capacity for the MAC data rate of 40 Gb/s. The 100GBASE-R PCS has a nominal rate at the PMA/FEC service interface of 5.15625 Gtransfers/s per PCS lane, which provides capacity for the MAC data rate of 100 Gb/s.

It is important to note that, while this specification defines interfaces in terms of bits, octets, and frames, implementations may choose other data-path widths for implementation convenience.

82.1.4.1 PCS service interface (XLGMII/CGMII)

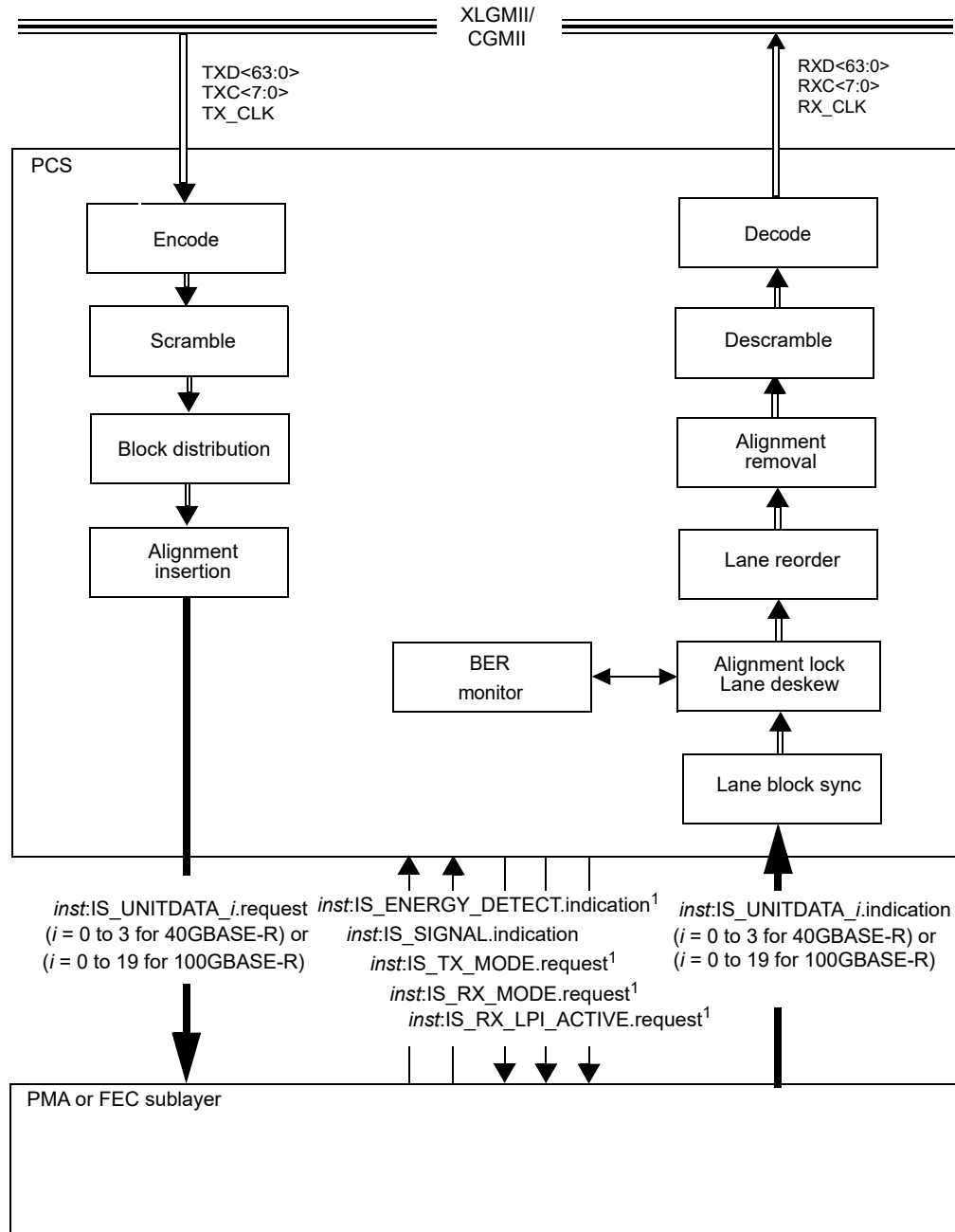
The PCS service interface allows the 40GBASE-R or 100GBASE-R PCS to transfer information to and from a PCS client. The PCS client is the Reconciliation Sublayer. The PCS Service Interface is precisely defined as the Media Independent Interface (XLGMII/CGMII) in Clause 81.

82.1.4.2 Physical Medium Attachment (PMA) or forward error correction (FEC) service interface

The PMA or FEC service interface for the PCS is described in an abstract manner and does not imply any particular implementation. The PMA/FEC Service Interface supports the exchange of encoded data between the PCS and PMA or FEC sublayer. The PMA or FEC service interface is defined in 83.3 or 94.2.1 and is an instance of the inter-sublayer service interface definition in 80.3 or 91.2.

82.1.5 Functional block diagram

Figure 82–2 provides a functional block diagram of the 40GBASE-R PCS and 100GBASE-R PCS.



NOTE 1—FOR OPTIONAL EEE DEEP SLEEP CAPABILITY

Figure 82–2—Functional block diagram

82.2 Physical Coding Sublayer (PCS)

82.2.1 Functions within the PCS

The 40GBASE-R and 100GBASE-R PCSs comprise the PCS Transmit and PCS Receive processes for each rate of operation. The PCS shields the Reconciliation Sublayer (and MAC) from the specific nature of the underlying channel. The PCS transmit channel and receive channel can each operate in normal mode or test-pattern mode.

When communicating with the XLGMII/CGMII, the PCS uses an eight octet-wide, synchronous data path, with packet delimiting being provided by transmit control signals ($\text{TXC}_{<n>} = 1$) and receive control signals ($\text{RXC}_{<n>} = 1$). When communicating with the PMA, the PCS uses multiple serial streams, 4 encoded bit streams for a 40GBASE-R PCS, or 20 encoded bit streams for a 100GBASE-R PCS. Alignment to 64B/66B blocks is performed in the PCS. The PMA sublayer operates independently of block and packet boundaries. The PCS provides the functions necessary to map packets between the XLGMII/CGMII format and the PMA/FEC service interface format.

Note that these serial streams originate from a common clock in each direction, but may vary in phase and Skew dynamically.

When the transmit channel is in normal mode, the PCS Transmit process continuously generates blocks based upon the $\text{TXD}_{<63:0>}$ and $\text{TXC}_{<7:0>}$ signals on the XLGMII/CGMII. The blocks are scrambled and then distributed to individual PCS lanes. After distribution, alignment marker blocks are periodically added to each PCS lane. Transmit data-units are sent to the service interface via the *inst:IS_UNITDATA_i.request* primitive. In these primitives, the *inst* can be either FEC or PMA depending on which sublayer is adjacent to the PCS sublayer.

When the transmit channel is in test-pattern mode, a test pattern is packed into the transmit data-units that are sent to the PMA/FEC service interface via the *inst:IS_UNITDATA_i.request* primitive.

When the receive channel is in normal or test-pattern mode, the PCS Synchronization process continuously monitors *inst:IS_SIGNAL.indication(SIGNAL_OK)*. When *SIGNAL_OK* indicates OK, then the PCS Synchronization process accepts data-units via the *inst:IS_UNITDATA_i.indication* primitive. It attains block synchronization based on the 2-bit synchronization headers on each one of the PCS lanes. Once block synchronization is found on a PCS lane, then alignment marker lock can be attained by searching for valid alignment markers. After alignment markers are found on all PCS lanes, the PCS lanes can be reordered and deskewed. Note that a particular transmit PCS lane can be received on any receive lane of the service interface due to the Skew and multiplexing that occurs in the path.

The PCS deskew process conveys received blocks to the PCS Receive process. The PCS deskew process deskews and aligns the individual PCS lanes, removes the alignment markers, forms a single stream, and sets the *align_status* flag to indicate whether the PCS has obtained alignment.

When the PCS deskew process has obtained alignment, the BER monitor process monitors the signal quality asserting *hi_ber* if excessive errors are detected. When *align_status* is asserted and *hi_ber* is deasserted, the PCS Receive process continuously accepts blocks and generates $\text{RXD}_{<63:0>}$ and $\text{RXC}_{<7:0>}$ on the MII.

When the receive channel is in test-pattern mode, the BER monitor process may be disabled. The Receive process will be held in the *RX_INIT* state. The received bits will be compared to the test pattern and errors counted.

The PCS shall provide transmit test-pattern mode for the scrambled idle pattern (see 82.2.11), and shall provide receive test-pattern mode for the scrambled idle pattern (see 82.2.11). Test-pattern mode is activated

separately for transmit and receive. The PCS shall support transmit test-pattern mode and receive test-pattern mode operating simultaneously so as to support loopback testing.

82.2.2 Use of blocks

The PCS maps XLGMII/CGMII signals into 66-bit blocks, and vice versa, using a 64B/66B coding scheme. The synchronization headers of the blocks allow establishment of block boundaries by the PCS Synchronization process. The PCS functions ENCODE and DECODE generate, manipulate, and interpret blocks as defined in 82.2.3.

82.2.3 64B/66B transmission code

The PCS uses a transmission code to improve the transmission characteristics of information to be transferred across the link and to support transmission of control and data characters. The encodings defined by the transmission code ensure that sufficient transitions are present in the PHY bit stream to make clock recovery possible at the receiver. The encoding also preserves the likelihood of detecting any single or multiple bit errors that may occur during transmission and reception of information. In addition, the synchronization headers of the code enable the receiver to achieve block alignment on the incoming PHY bit stream. The 64B/66B transmission code specified for use in this standard is a run-length-limited code.¹⁴⁵

The relationship of block bit positions relative to XLGMII/CGMII, PMA, and other PCS functions is illustrated in Figure 82–3 for transmit and Figure 82–4 for receive. These figures illustrate the processing of a block containing 8 data octets. See 82.2.3.3 for information on how blocks containing control characters are mapped into 66-bit blocks. Note that the sync header is generated by the encoder and bypasses the scrambler.

82.2.3.1 Notation conventions

For values shown as binary, the leftmost bit is the first transmitted bit.

64B/66B encodes 8 data octets or control characters into a block. Blocks containing control characters also contain a block type field. Data octets are labeled D_0 to D_7 . The control characters /I/ and /E/ are labeled C_0 to C_7 . The control character /Q/ for a sequence ordered set is labeled as O_0 since it is only valid on the first octet of the XLGMII/CGMII. The control character for start is labeled as S_0 for the same reason. The control character for terminate is labeled as T_0 to T_7 . The four trailing zero data octets in ordered sets are labeled as Z_4 to Z_7 .

One XLGMII/CGMII transfer provides eight characters that are encoded into one 66-bit transmission block. The subscript in the above labels indicates the position of the character in the eight characters from the XLGMII/CGMII transfer.

Contents of block type fields, data octets, and control characters are shown as hexadecimal values. The LSB of the hexadecimal value represents the first transmitted bit. For instance, the block type field 0x1E is sent from left to right as 01111000. The bits of a transmitted or received block are labeled $TxB<65:0>$ and $RxB<65:0>$, respectively, where $TxB<0>$ and $RxB<0>$ represent the first transmitted bit. The value of the sync header is shown as a binary value. Binary values are shown with the first transmitted bit (the LSB) on the left.

¹⁴⁵In 10GBASE-R the run length is limited to 66 bits, but in 40GBASE-R and 100GBASE-R, when multiplexing occurs in the PMA, this guaranteed run length limit increases. For example, if two PCS lanes are multiplexed in the PMA, then the possible run length would double.

82.2.3.2 Transmission order

Block bit transmission order shall be as illustrated in Figure 82–3 and Figure 82–4. Note that these figures show the mapping from XLGMII/CGMII to 64B/66B block for a block containing eight data characters.

82.2.3.3 Block structure

Blocks consist of 66 bits. The first two bits of a block are the synchronization header (sync header). Blocks are either data blocks or control blocks. The sync header is 01 for data blocks and 10 for control blocks. Thus, there is always a transition between the first two bits of a block. The remainder of the block contains the payload. The payload is scrambled and the sync header bypasses the scrambler. Therefore, the sync header is the only position in the block that is always guaranteed to contain a transition. This feature of the code is used to obtain block synchronization.

Data blocks contain eight data characters. Control blocks begin with an 8-bit block type field that indicates the format of the remainder of the block. For control blocks containing a Start, Terminate character, or ordered set, that character is implied by the block type field. Other control characters are encoded in a 7-bit control code. Each control block encodes eight characters.

The format of the blocks is as shown in Figure 82–5. In the figure, the column labeled Input Data shows, in abbreviated form, the eight characters used to create the 66-bit block. These characters are either data characters or control characters and, when transferred across the XLGMII/CGMII, the corresponding TXC or RXC bit is set accordingly. Within the Input Data column, D_0 through D_7 are data octets and are transferred with the corresponding TXC or RXC bit set to zero. All other characters are control characters and are transferred with the corresponding TXC or RXC bit set to one. The single bit fields (thin rectangles with no label in the figure) are sent as zero and ignored upon receipt.

All unused values of block type field¹⁴⁶ are invalid; they shall not be transmitted and shall be considered an error if received.

¹⁴⁶The block type field values have been chosen to have a 4-bit Hamming distance between them. There are five unused values that maintain this Hamming distance: 0x00, 0x2D, 0x33, 0x55, and 0x66.

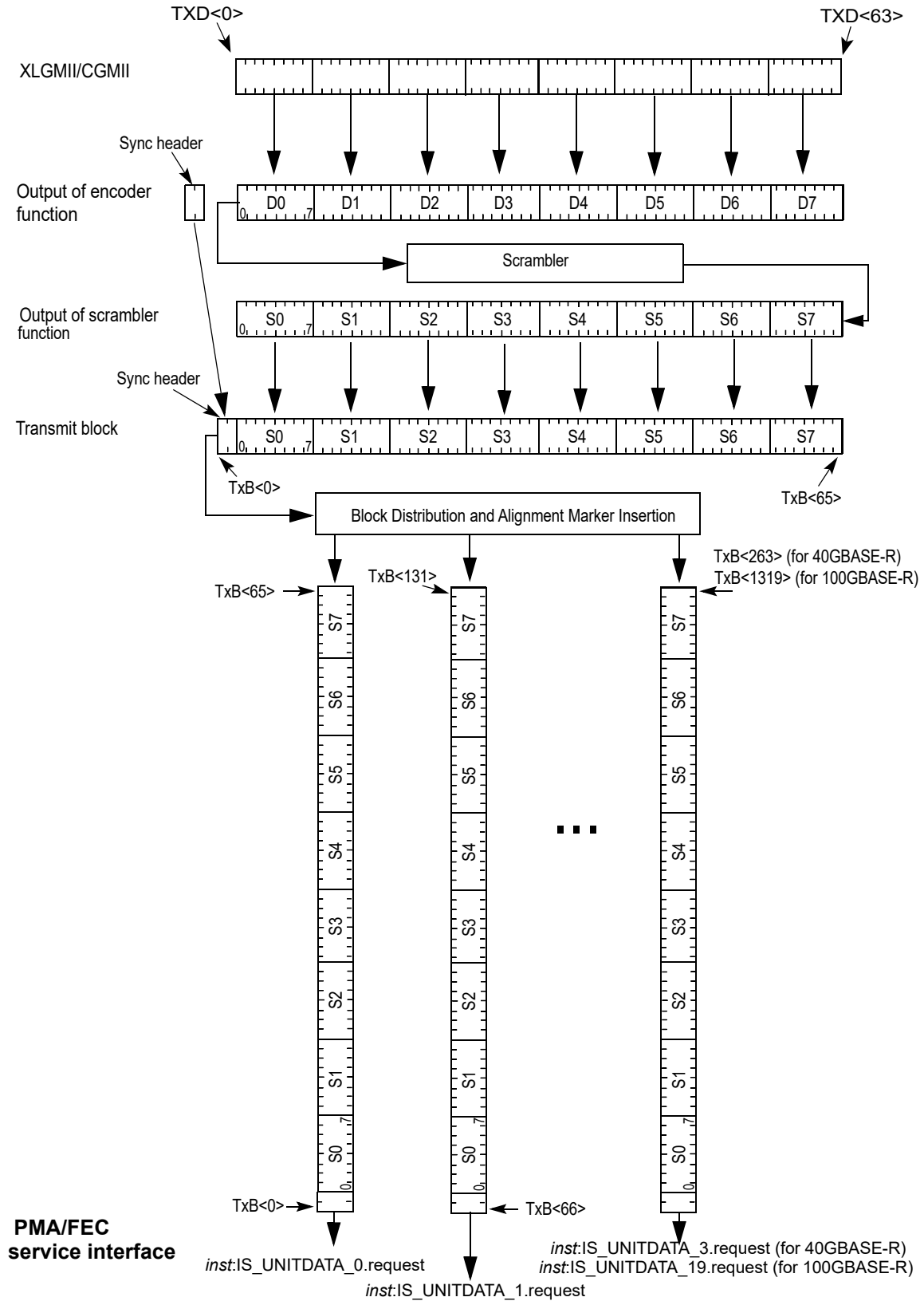


Figure 82–3—PCS Transmit bit ordering

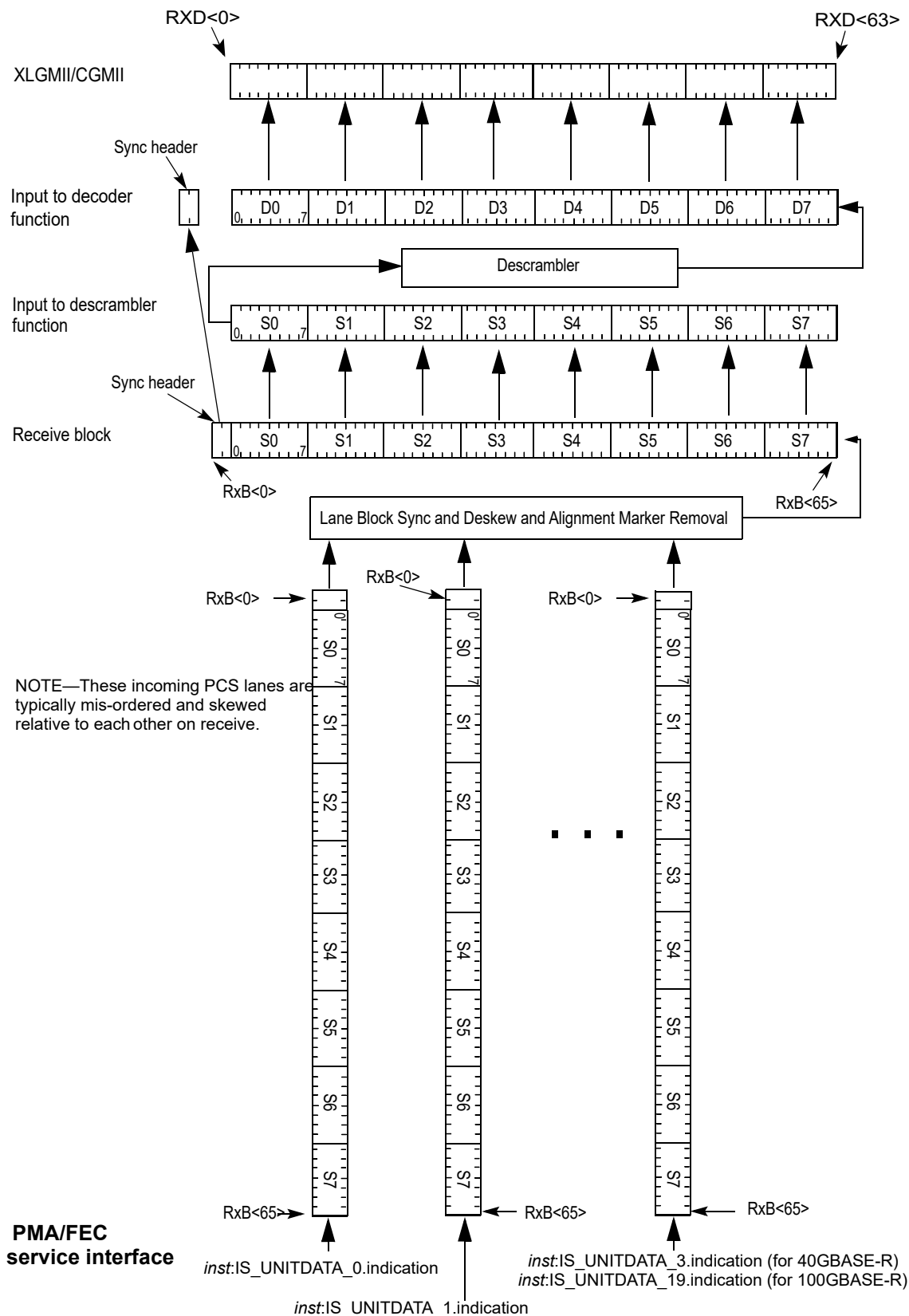


Figure 82-4—PCS Receive bit ordering

Input Data	S y n c	Block Payload											
Bit Position: Data Block Format:	0 1 2	65											
D ₀ D ₁ D ₂ D ₃ D ₄ D ₅ D ₆ D ₇	01	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇				
Control Block Formats:		Block Type Field											
C ₀ C ₁ C ₂ C ₃ C ₄ C ₅ C ₆ C ₇	10	0x1E	C ₀	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇			
S ₀ D ₁ D ₂ D ₃ D ₄ D ₅ D ₆ D ₇	10	0x78	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇				
O ₀ D ₁ D ₂ D ₃ Z ₄ Z ₅ Z ₆ Z ₇	10	0x4B	D ₁	D ₂	D ₃	O ₀	0x000_0000						
T ₀ C ₁ C ₂ C ₃ C ₄ C ₅ C ₆ C ₇	10	0x87					C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
D ₀ T ₁ C ₂ C ₃ C ₄ C ₅ C ₆ C ₇	10	0x99	D ₀				C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	
D ₀ D ₁ T ₂ C ₃ C ₄ C ₅ C ₆ C ₇	10	0xAA	D ₀	D ₁			C ₃	C ₄	C ₅	C ₆	C ₇		
D ₀ D ₁ D ₂ T ₃ C ₄ C ₅ C ₆ C ₇	10	0xB4	D ₀	D ₁	D ₂			C ₄	C ₅	C ₆	C ₇		
D ₀ D ₁ D ₂ D ₃ T ₄ C ₅ C ₆ C ₇	10	0xCC	D ₀	D ₁	D ₂	D ₃			C ₅	C ₆	C ₇		
D ₀ D ₁ D ₂ D ₃ D ₄ T ₅ C ₆ C ₇	10	0xD2	D ₀	D ₁	D ₂	D ₃	D ₄			C ₆	C ₇		
D ₀ D ₁ D ₂ D ₃ D ₄ D ₅ T ₆ C ₇	10	0xE1	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅		C ₇			
D ₀ D ₁ D ₂ D ₃ D ₄ D ₅ D ₆ T ₇	10	0xFF	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆				

Figure 82–5—64B/66B block formats

WARNING

The mapping of 40GBASE-R PCS blocks into OPU3 specified in ITU-T G.709 depends on the set of control block types shown in Figure 82–5. Any deviation from the coding specified in Figure 82–5 will break the mapping and may prevent 40GBASE-R PCS blocks from being mapped into OPU3 (see ITU-T G.709 for more details).

82.2.3.4 Control codes

The same set of control characters are supported by the XLGMII/CGMII and the PCS. The representations of the control characters are the control codes. XLGMII/CGMII encodes a control character into an octet (an eight bit value). The 40GBASE-R and 100GBASE-R PCS encode the start and terminate control characters implicitly by the block type field. The 40GBASE-R and 100GBASE-R PCS encode the ordered set control codes using the block type field. The 40GBASE-R and 100GBASE-R PCS encode each of the other control characters into a 7-bit control code.

The control characters and their mappings to 40GBASE-R and 100GBASE-R control codes and XLGMII/CGMII control codes are specified in Table 82–1. All XLGMII/CGMII, 40GBASE-R, and 100GBASE-R control code values that do not appear in the table shall not be transmitted and shall be considered an error if received. The ability to transmit or receive Low Power Idle (LPI) is required for PHYs that support EEE (see Clause 78). If EEE has not been negotiated, LPI shall not be transmitted and shall be treated as an error if received.

82.2.3.5 Valid and invalid blocks

A block is invalid if any of the following conditions exists:

- a) The sync header has a value of 00 or 11.
- b) The block type field contains an invalid value (one not included in Figure 82–5).
- c) Any control character contains a value not included in Table 82–1.
- d) The set of eight XLGMII/CGMII characters does not have a corresponding block format in Figure 82–5.

82.2.3.6 Idle (/I/)

Idle control characters (/I/) are transmitted when idle control characters are received from the XLGMII/CGMII. Idle control characters may be added or deleted by the PCS to adapt between clock rates. /I/ insertion and deletion shall occur in groups of 8. /I/s may be added following idle control characters or ordered sets. They shall not be added while data is being received.

Table 82–1—Control codes

Control character	Notation	XLGMII/ CGMII control code	40/100GBASE-R O code ^a	40GBASE-R and 100GBASE-R control code
idle	/I/	0x07		0x00
LPI	/LI/	0x06		0x06
start	/S/	0xFB		Encoded by block type field
terminate	/T/	0xFD		Encoded by block type field
error	/E/	0xFE		0x1E
Sequence ordered set	/Q/	0x9C	0x0	Encoded by block type 0x4B plus O code, control codes are set to 0x00

^a INCITS T11 Fibre Channel uses O code 0xF for the signal ordered set. OIF uses O code 0x5 for the FlexE [B59] ordered set. ITU-T uses O code 0xC for the G.8312 [B47] MTN path ordered set.

To communicate LPI, the LPI control character /LI/ is sent continuously in place of /I/. LPI control characters are transmitted when LPI control characters are received from the XLGMII or CGMII. LPI characters may be added or deleted by the PCS to adapt between clock rates in a similar manner to idle control characters. /LI/ insertion and deletion shall occur in groups of 8. /LI/s inserted for clock compensation may only be inserted following other LPI characters.

82.2.3.7 Start (/S/)

The start control character (/S/) indicates the start of a packet. This delimiter is only valid on the first octet of the XLGMII/CGMII (TXD<7:0> and RXD<7:0>). Receipt of an /S/ on any other octet of TXD indicates an error. Block type field values implicitly encode an /S/ as the first character of the block.

82.2.3.8 Terminate (/T/)

The terminate control character (/T/) indicates the end of a packet. Since packets vary in length, the /T/ can occur on any octet of the XLGMII/CGMII and on any character of the block. The location of the /T/ in the block is implicitly encoded in the block type field. A valid end of packet occurs when a block containing a /T/ is followed by a control block that does not contain a /T/ or an /E/.

82.2.3.9 ordered set (/O/)

Ordered sets are used to extend the ability to send control and status information over the link such as remote fault and local fault status. Ordered sets consist of a control character followed by three data characters followed by four zero data characters on the XLGMII/CGMII. Ordered sets always begin on the first octet of the XLGMII/CGMII. 40 Gigabit and 100 Gigabit Ethernet use one kind of ordered set: the sequence ordered set (see 81.3.4). The ordered set control character /Q/ indicates the start of an ordered set. The block type field plus the O code encode the specific control character for the ordered set. See Table 82–1 for the mapping.

Sequence ordered sets may be deleted by the PCS to adapt between clock rates. Such deletion shall only occur when two consecutive sequence ordered sets have been received and only one of the two ordered sets may be deleted. Only idle control characters may be inserted for clock compensation.

82.2.3.10 Error (/E/)

In both the 64B/66B encoder and decoder, the /E/ is generated whenever an /E/ is detected. The /E/ is also generated when invalid blocks are detected. The /E/ allows the PCS to propagate detected errors. See R_TYPE and T_TYPE function definitions in 82.2.19.2.3 for further information.

82.2.4 Transmit process

The transmit process generates blocks based upon the TXD<63:0> and TXC<7:0> signals received from the XLGMII/CGMII. One XLGMII/CGMII data transfer is encoded into one 66-bit block. It takes 66 *inst*:IS_UNITDATA_i transfers to send a 66-bit block of data on each of the PCS lanes. The transmit process may delete idle control characters or sequence ordered sets to accommodate the transmission of alignment markers. If the PCS transmit process spans multiple clock domains, it may also perform clock rate compensation via the deletion of idle control characters or sequence ordered sets or the insertion of idle control characters.

There are sufficient idle control characters to delete in order to make room for alignment markers, in addition to handling clock compensation. Idle control characters or sequence ordered sets are removed, if necessary, to accommodate the insertion of the 66-bit alignment markers. See 82.2.3.6 for more details.

The transmit process generates blocks as specified in the transmit process state diagram. The contents of each block are contained in a vector tx_coded<65:0>, which is passed to the scrambler. tx_coded<1:0> contains the sync header and the remainder of the bits contain the block payload.

82.2.5 Scrambler

The payload, tx_coded<65:2>, is scrambled with a self-synchronizing scrambler. The scrambler is identical to the scrambler used in Clause 49, see 49.2.6 for the definition of the scrambler. The sync bits, tx_coded<1:0>, are not scrambled.

82.2.6 Block distribution

Once the data is encoded and scrambled, it is distributed to multiple PCS lanes, 66-bit blocks at a time in a round robin distribution from the lowest to the highest numbered PCS lanes. This allows the PCS to support multiple physical lanes in the PMD and XLAUI or CAUI-n interfaces (see Annex 83A, Annex 83B, Annex 83D, and Annex 83E). The 40GBASE-R PCS distributes the 66-bit blocks to 4 PCS lanes, and the 100GBASE-R PCS distributes the blocks to 20 PCS lanes. The distribution process is shown in Figure 82–6.

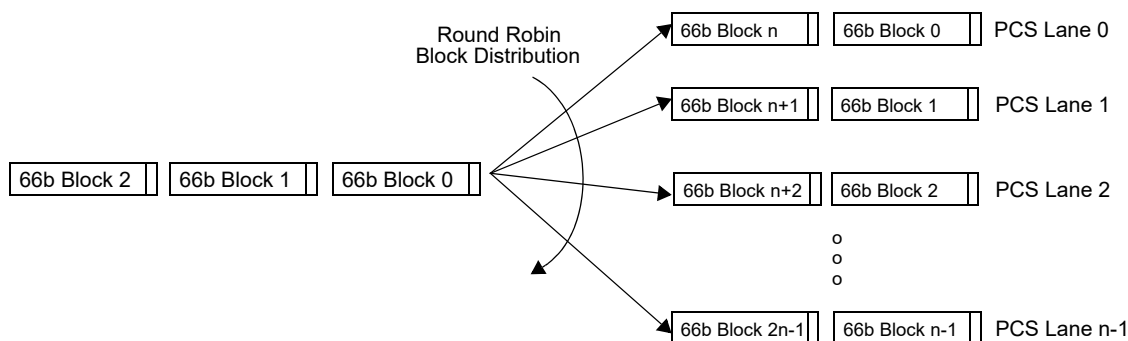


Figure 82–6—PCS Block distribution

82.2.7 Alignment marker insertion

In order to support deskew and reordering of individual PCS lanes at the receive PCS, alignment markers are added periodically to each PCS lane. The alignment marker has the form of a specially defined 66-bit block with a control block sync header. These markers interrupt any data transfer that is already in progress. This allows alignment markers to be inserted into all PCS lanes at the same time. If necessary, room for the alignment markers is created by periodically deleting IPG from the XLGMII/CGMII data stream. Other special properties of the alignment markers are that they are not scrambled and do not conform to the encoding rules as outlined in Figure 82–5. This is possible because the alignment markers are added after encoding is performed in the transmit PCS and the alignment markers are removed before 64B/66B decoding is performed in the receive PCS. The alignment markers are not scrambled in order to allow the receiver to find the alignment markers, deskew the PCS lanes, and reassemble the aggregate stream before descrambling is performed. The alignment markers themselves are formed from a known pattern that is defined to be balanced and with many transitions and therefore scrambling is not necessary for the alignment markers. The alignment markers shall be inserted after every 16383 66-bit blocks on each PCS lane. Alignment marker insertion is shown in Figure 82–7 and Figure 82–8.

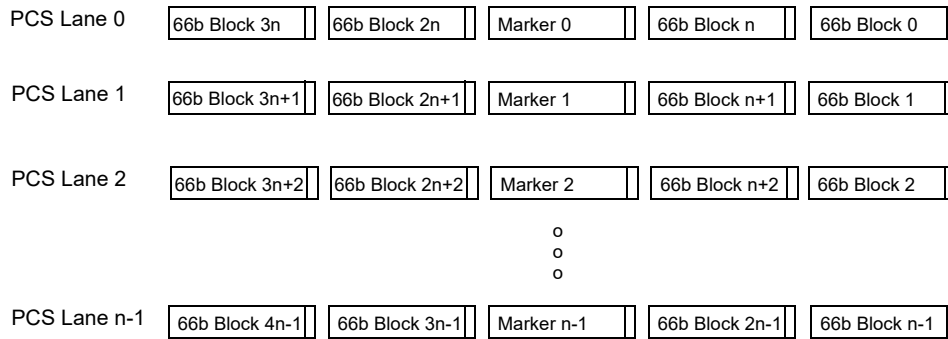


Figure 82-7—Alignment marker insertion

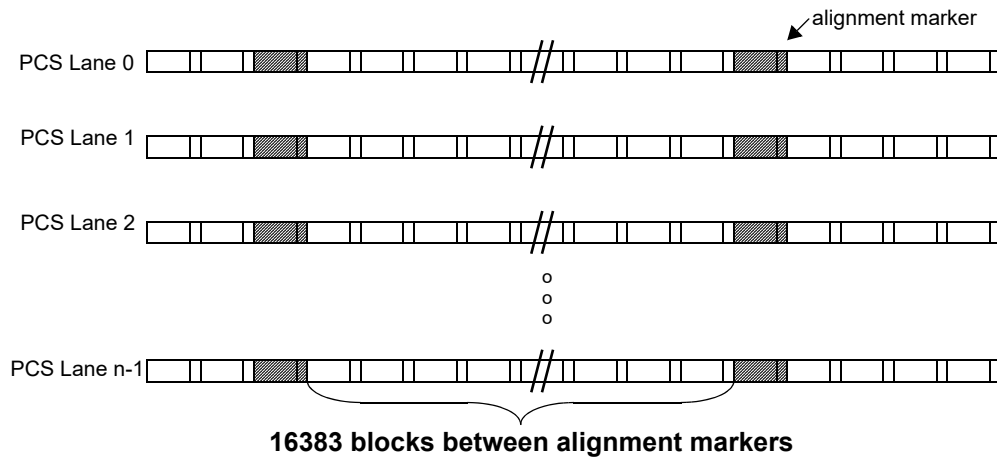


Figure 82–8—Alignment marker insertion period

The format of the alignment markers is shown in Figure 82–9.

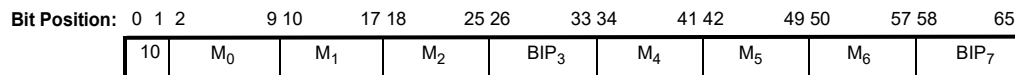


Figure 82–9—Alignment marker format

The content of the alignment markers shall be as shown in Table 82–2 for 100GBASE-R and in Table 82–3 for 40GBASE-R. The contents depend on the PCS lane number and the octet number. Note that M_4 through M_6 are the bit-wise inversion of M_0 through M_2 , respectively. Also BIP_7 is the bit-wise inversion of BIP_3 . This property allows the alignment markers to be DC-balanced. Lane markers 0 to 19 from Table 82–2 are used for the 100GBASE-R PCS and lane markers 0 to 3 from Table 82–3 are used for the 40GBASE-R PCS. As an example, the lane marker for 100GBASE-R lane number 0 is sent as (left most bit sent first):

10 10000011 00010110 10000100 BIP₃ 01111100 11101001 01111011 BIP₇

After the alignment markers are inserted, data is sent to the PMA or FEC sublayer adjacent to the PCS.

Table 82–2—100GBASE-R Alignment marker encodings

PCS lane number	Encoding ^a {M ₀ , M ₁ , M ₂ , BIP ₃ , M ₄ , M ₅ , M ₆ , BIP ₇ }	PCS lane number	Encoding ^a {M ₀ , M ₁ , M ₂ , BIP ₃ , M ₄ , M ₅ , M ₆ , BIP ₇ }
0	0xC1, 0x68, 0x21, BIP ₃ , 0x3E, 0x97, 0xDE, BIP ₇	10	0xFD, 0x6C, 0x99, BIP ₃ , 0x02, 0x93, 0x66, BIP ₇
1	0x9D, 0x71, 0x8E, BIP ₃ , 0x62, 0x8E, 0x71, BIP ₇	11	0xB9, 0x91, 0x55, BIP ₃ , 0x46, 0x6E, 0xAA, BIP ₇
2	0x59, 0x4B, 0xE8, BIP ₃ , 0xA6, 0xB4, 0x17, BIP ₇	12	0x5C, 0xB9, 0xB2, BIP ₃ , 0xA3, 0x46, 0x4D, BIP ₇
3	0x4D, 0x95, 0x7B, BIP ₃ , 0xB2, 0x6A, 0x84, BIP ₇	13	0x1A, 0xF8, 0xBD, BIP ₃ , 0xE5, 0x07, 0x42, BIP ₇
4	0xF5, 0x07, 0x09, BIP ₃ , 0x0A, 0xF8, 0xF6, BIP ₇	14	0x83, 0xC7, 0xCA, BIP ₃ , 0x7C, 0x38, 0x35, BIP ₇
5	0xDD, 0x14, 0xC2, BIP ₃ , 0x22, 0xEB, 0x3D, BIP ₇	15	0x35, 0x36, 0xCD, BIP ₃ , 0xCA, 0xC9, 0x32, BIP ₇
6	0x9A, 0x4A, 0x26, BIP ₃ , 0x65, 0xB5, 0xD9, BIP ₇	16	0xC4, 0x31, 0x4C, BIP ₃ , 0x3B, 0xCE, 0xB3, BIP ₇
7	0x7B, 0x45, 0x66, BIP ₃ , 0x84, 0xBA, 0x99, BIP ₇	17	0xAD, 0xD6, 0xB7, BIP ₃ , 0x52, 0x29, 0x48, BIP ₇
8	0xA0, 0x24, 0x76, BIP ₃ , 0x5F, 0xDB, 0x89, BIP ₇	18	0x5F, 0x66, 0x2A, BIP ₃ , 0xA0, 0x99, 0xD5, BIP ₇
9	0x68, 0xC9, 0xFB, BIP ₃ , 0x97, 0x36, 0x04, BIP ₇	19	0xC0, 0xF0, 0xE5, BIP ₃ , 0x3F, 0x0F, 0x1A, BIP ₇

^aEach octet is transmitted LSB to MSB.

Table 82–3—40GBASE-R Alignment marker encodings

PCS lane number	Encoding ^a {M ₀ , M ₁ , M ₂ , BIP ₃ , M ₄ , M ₅ , M ₆ , BIP ₇ }
0	0x90, 0x76, 0x47, BIP ₃ , 0x6F, 0x89, 0xB8, BIP ₇
1	0xF0, 0xC4, 0xE6, BIP ₃ , 0x0F, 0x3B, 0x19, BIP ₇
2	0xC5, 0x65, 0x9B, BIP ₃ , 0x3A, 0x9A, 0x64, BIP ₇
3	0xA2, 0x79, 0x3D, BIP ₃ , 0x5D, 0x86, 0xC2, BIP ₇

^aEach octet is transmitted LSB to MSB.

82.2.8 BIP calculations

A PCS lane BIP field is carried in each PCS Lane alignment marker. This allows an accurate and fast measure of the bit error ratio of a given PCS Lane. This information is used to update error counters; no state machines use this information.

Each alignment marker has two Bit Interleaved Parity fields, BIP₃ and BIP₇. BIP₇ is a bit-wise inversion of BIP₃ in order to keep the alignment marker DC-balanced. The BIP₃ field contains the result of a bit interleaved parity calculation. Each bit in the BIP field is an even parity calculation over all of the previous specified bits of a given PCS Lane, from and including the previous alignment marker, but not including the current alignment marker. Using the bit definitions as shown in Figure 82–9, Table 82–4 has the bit assignments for each BIP₃ bit. As an example, BIP₃ bit 0 contains the result of XORing 131072 bits from 16384 66-bit words. BIP₃ bit 3 and bit 4 also include one sync header bit from each 66-bit word. Bit 3 and bit 4 each contain the result of XORing 147456 bits.

Table 82–4—BIP₃ bit assignments

BIP ₃ bit number	Assigned 66-bit word bits
0	2, 10, 18, 26, 34, 42, 50, 58
1	3, 11, 19, 27, 35, 43, 51, 59
2	4, 12, 20, 28, 36, 44, 52, 60
3	0, 5, 13, 21, 29, 37, 45, 53, 61
4	1, 6, 14, 22, 30, 38, 46, 54, 62
5	7, 15, 23, 31, 39, 47, 55, 63
6	8, 16, 24, 32, 40, 48, 56, 64
7	9, 17, 25, 33, 41, 49, 57, 65

BIP₃ and BIP₇ are transmitted LSB to MSB. As an example, with BIP₃ = 0x0F, the PCS lane marker for 100GBASE-R lane number 0 is sent as (left most bit sent first):

10 10000011 00010110 10000100 11110000 01111100 11101001 01111011 00001111

If the EEE capability is supported, BIP statistics are only updated when the receiver is in the RX_ACTIVE state (see Figure 82–19). In all other states, the running parity is not calculated. The BIP statistics shall be first updated after transitioning from RAMs to normal AMs on the first received normal AM when LPI_FW is FALSE and on the second received AM after entering the RX_ACTIVE state when LPI_FW is TRUE.

82.2.9 Rapid alignment marker insertion

For the optional EEE capability, an alternate method of alignment is used when operating in the deep sleep low power state. Rapid Alignment Markers (RAMs) function in a similar manner to the alignment markers described in 82.2.7. Normal alignment markers are sent when the transmitter has an LPI transmit state of TX_ACTIVE; RAMs are sent in the TX_WAKE2 state until down_count_done is TRUE and in all the other LPI transmit states. Additionally, the BIP component defined for alignment markers is replaced by a count

down field (CD) so that the transition from RAMs to normal alignment markers can be indicated. The RAMs shall be inserted after every 7 66-bit blocks on each 100G PCS lane and every 15 66-bit blocks on each 40G PCS lane. RAM insertion is performed in the same manner as shown in Figure 82–7 and Figure 82–10. The transition from RAMs to normal alignment markers is shown in Figure 82–10. The count down field is also used to communicate some of the states of the tx_mode when it is not being used to coordinate the transition. After the LPI Transmit state diagram transitions from TX_ACTIVE to TX_SLEEP, the first RAM shall be inserted after at least one block of /LI/ has been transmitted on PCS lane 0. In order to force the RAMs to coincide with the start of an FEC block, the distance between the first RAM and preceding normal alignment marker shall be a multiple of 4 66-bit blocks.

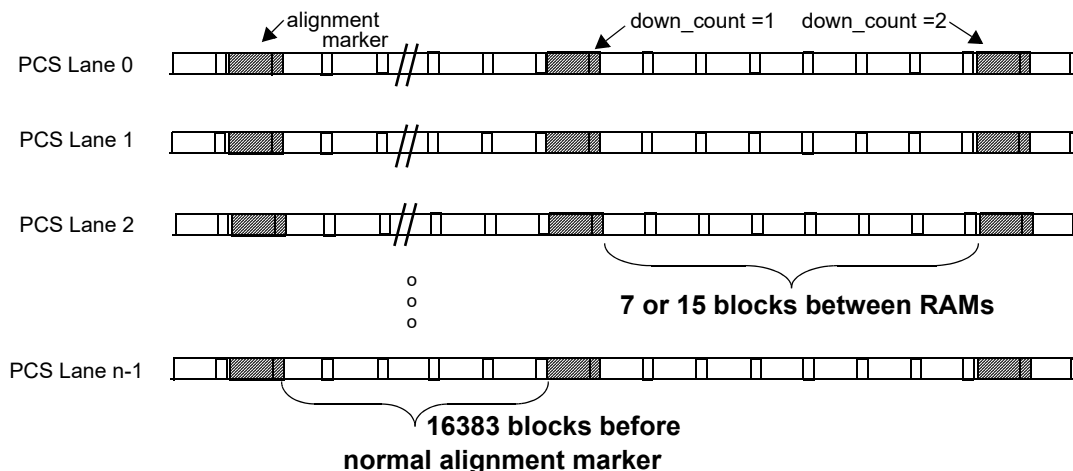


Figure 82–10—RAM transition

The format of the RAMs is shown in Figure 82–11.

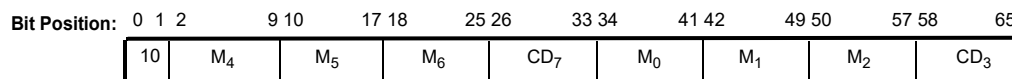


Figure 82–11—RAM format

The content of the RAMs shall be as shown in Table 82–5 for 100GBASE-R or Table 82–6 for 40GBASE-R. Note that these are similar to normal alignment markers, with CD₃ replacing BIP₃ and CD₇ replacing BIP₇, and also M₀ through M₂ and CD₃ swapped with M₄ through M₆ and CD₇, respectively. As an example, the lane marker for 100GBASE-R lane number 0 is sent as (left most bit sent first):

10 01111100 11101001 01111011 CD₇ 10000011 00010110 10000100 CD₃

After the RAMs are inserted, data is sent to the PMA or FEC sublayer adjacent to the PCS.

The value of the CD₃ field is derived by the bit-wise XOR of the down_count variable with the M₀ value for the lane (therefore the last 5 RAMs sent by a 100GBASE-R PCS on PCS lane 0 would have CD₃ values: 0xC4, 0xC5, 0xC2, 0xC3, 0xC0; for PCS lane 1 these would be: 0x98, 0x99, 0x9E, 0x9F, 0x9C). The CD₇ field is the bit-wise inversion of CD₃. The CD field is used by the link partner to understand the expected

transition from RAMs to normal AMs. It may also be used by a device with a detached PMA or FEC sublayer to infer the state of the PCS.

Table 82–5—100GBASE-R RAM encodings

PCS lane number	Encoding ^a {M ₄ , M ₅ , M ₆ , CD ₇ , M ₀ , M ₁ , M ₂ , CD ₃ }	PCS lane number	Encoding ^a {M ₄ , M ₅ , M ₆ , CD ₇ , M ₀ , M ₁ , M ₂ , CD ₃ }
0	0x3E, 0x97, 0xDE, CD ₇ , 0xC1, 0x68, 0x21, CD ₃	10	0x02, 0x93, 0x66, CD ₇ , 0xFD, 0x6C, 0x99, CD ₃
1	0x62, 0x8E, 0x71, CD ₇ , 0x9D, 0x71, 0x8E, CD ₃	11	0x46, 0x6E, 0xAA, CD ₇ , 0xB9, 0x91, 0x55, CD ₃
2	0xA6, 0xB4, 0x17, CD ₇ , 0x59, 0x4B, 0xE8, CD ₃	12	0xA3, 0x46, 0x4D, CD ₇ , 0x5C, 0xB9, 0xB2, CD ₃
3	0xB2, 0x6A, 0x84, CD ₇ , 0x4D, 0x95, 0x7B, CD ₃	13	0xE5, 0x07, 0x42, CD ₇ , 0x1A, 0xF8, 0xBD, CD ₃
4	0x0A, 0xF8, 0xF6, CD ₇ , 0xF5, 0x07, 0x09, CD ₃	14	0x7C, 0x38, 0x35, CD ₇ , 0x83, 0xC7, 0xCA, CD ₃
5	0x22, 0xEB, 0x3D, CD ₇ , 0xDD, 0x14, 0xC2, CD ₃	15	0xCA, 0xC9, 0x32, CD ₇ , 0x35, 0x36, 0xCD, CD ₃
6	0x65, 0xB5, 0xD9, CD ₇ , 0x9A, 0x4A, 0x26, CD ₃	16	0x3B, 0xCE, 0xB3, CD ₇ , 0xC4, 0x31, 0x4C, CD ₃
7	0x84, 0xBA, 0x99, CD ₇ , 0x7B, 0x45, 0x66, CD ₃	17	0x52, 0x29, 0x48, CD ₇ , 0xAD, 0xD6, 0xB7, CD ₃
8	0x5F, 0xDB, 0x89, CD ₇ , 0xA0, 0x24, 0x76, CD ₃	18	0xA0, 0x99, 0xD5, CD ₇ , 0x5F, 0x66, 0x2A, CD ₃
9	0x97, 0x36, 0x04, CD ₇ , 0x68, 0xC9, 0xFB, CD ₃	19	0x3F, 0x0F, 0x1A, CD ₇ , 0xC0, 0xF0, 0xE5, CD ₃

^aEach octet is transmitted LSB to MSB.

Table 82–6—40GBASE-R RAM encodings

PCS lane number	Encoding ^a {M ₄ , M ₅ , M ₆ , CD ₇ , M ₀ , M ₁ , M ₂ , CD ₃ }
0	0x6F, 0x89, 0xB8, CD ₇ , 0x90, 0x76, 0x47, CD ₃
1	0x0F, 0x3B, 0x19, CD ₇ , 0xF0, 0xC4, 0xE6, CD ₃
2	0x3A, 0x9A, 0x64, CD ₇ , 0xC5, 0x65, 0x9B, CD ₃
3	0x5D, 0x86, 0xC2, CD ₇ , 0xA2, 0x79, 0x3D, CD ₃

^aEach octet is transmitted LSB to MSB.

82.2.10 PMA or FEC Interface

When the transmit channel is operating in normal mode, the 40GBASE-R PCS sends four data streams via *inst:IS_UNITDATA_i.request* primitives and the 100GBASE-R PCS sends twenty data streams via *inst:IS_UNITDATA_i.request* primitives.

The *inst:IS_UNITDATA_i.request* primitives are separate serial streams of bits. Since 66-bit blocks were distributed to each lane, that means for the 40GBASE-R PCS: bits 0 to 65 are sent on PCSL 0, bits 66 to 131

are sent on PCSL 1; bits 132 to 197 are sent on PCSL 2, bits 198 to 263 are sent on PCSL 3, then bits 264 to 329 are sent on PCSL 0 etc.

For 100GBASE-R it is: bits 0 to 65 on PCSL 0, bits 66 to 131 on PCSL 1, bits 132 to 197 on PCSL 2, bits 198 to 263 on PCSL 3, bits 264 to 329 on PCSL 4, bits 330 to 395 on PCSL 5, bits 396 to 461 on PCSL 6, bits 462 to 527 on PCSL 7, bits 528 to 593 on PCSL 8, bits 594 to 659 on PCSL 9, bits 660 to 725 on PCSL 10, bits 726 to 791 on PCSL 11, bits 792 to 857 on PCSL 12, bits 858 to 923 on PCSL 13, bits 924 to 989 on PCSL 14, bits 990 to 1055 on PCSL 15, bits 1056 to 1121 on PCSL 16, bits 1122 to 1187 on PCSL 17, bits 1188 to 1253 on PCSL 18, bits 1254 to 1319 on PCSL 19, then bits 1320 to 1385 on PCSL 0, etc.

82.2.11 Test-pattern generators

The PCS shall have the ability to generate and detect a scrambled idle test pattern. This test-pattern mode is suitable for receiver tests and for certain transmitter tests.

When a scrambled idle pattern is enabled, the test pattern is generated by the scrambler. No seeding of the scrambler is required during test-pattern operation. The input to the scrambler is a control block (block type=0x1E) with all idles as defined in Figure 82–5. Note that the sync headers and alignment markers are added to the stream so that the receive PCS can align and deskew the PCS lanes.

If a Clause 45 MDIO is implemented, then control of the test-pattern generation is from the BASE-R PCS test-pattern control register (bit 3.42.3).

When the transmit channel is operating in test-pattern mode, the encoded bit stream is distributed to the PCS Lanes as in normal operation (see 82.2.4).

82.2.12 Block synchronization

When the receive channel is operating in normal mode, the block synchronization function receives data via 4 (for 40GBASE-R) or 20 (for 100GBASE-R) *IS_UNITDATA_i.indication* primitives. The PCS forms 4 or 20 bit streams from the primitives by concatenating the bits from the indications of each primitive in order from each *inst:IS_UNITDATA_0.indication* to *inst:IS_UNITDATA_3.indication* or *inst:IS_UNITDATA_0.indication* to *inst:IS_UNITDATA_19.indication*. It obtains lock to the 66-bit blocks in each bit stream using the sync headers and outputs 66-bit blocks. Block lock is obtained as specified in the block lock state diagram shown in Figure 82–12.

If EEE is not supported then *block_lock* is identical to *rx_block_lock*. Otherwise the relationship between *block_lock* and *rx_block_lock* is given by Figure 82–19.

82.2.13 PCS lane deskew

Once the receiver achieves block lock on a lane, then it begins obtaining alignment marker lock as specified in the alignment marker lock state diagram shown in Figure 82–13. This process identifies the PCS lane number received on a particular lane of the service interface. After alignment marker lock is achieved on all lanes (4 or 20 lanes), then all inter-lane Skew is removed as shown in the PCS deskew state diagram in Figure 82–14. The Skew budget that the PCS receiver shall support is shown in Table 82–7.

Note that Skew is defined in 80.5.

Table 82–7—Skew tolerance requirements

PCS	Maximum Skew	Maximum Skew Variation
40GBASE-R	180 ns (~1856 bits)	4ns (~41 bits)
100GBASE-R	180 ns (~928 bits)	4ns (~21 bits)
100GBASE-R with RS-FEC	49 ns (~253 bits)	0.4 ns (~2 bits)

If EEE is not supported then align_status is identical to rx_align_status. Otherwise the relationship between align_status and rx_align_status is given by Figure 82–19.

82.2.14 PCS lane reorder

Transmit PCS lanes can be received on different lanes of the service interface from which they were originally transmitted due to Skew between lanes and multiplexing by the PMA. The receive PCS shall order the received PCS lanes according to the PCS lane number.

82.2.15 Alignment marker removal

After all PCS lanes are aligned and deskewed, the PCS lanes are multiplexed together in the proper order to reconstruct the original stream of blocks and the alignment markers are deleted from the data stream. If necessary, the difference in rate due to deleted alignment markers is compensated for by inserting idle control characters. Note that an alignment marker is always deleted when a given PCS Lane is in am_lock=true even if it does not match the expected alignment marker value (due to a bit error for example). Repeated alignment marker errors will result in am_lock being set to false for a given PCS Lane, but until that happens it is sufficient to delete the block in the alignment marker position.

As part of the alignment marker removal process, the BIP₃ field is compared to the calculated BIP value for each PCS lane. If a Clause 45 MDIO is implemented, then the appropriate BIP error counter register (registers 3.200 through 3.219) is incremented by one each time the calculated BIP value does not equal the value received in the BIP₃ field. The incoming bit error ratio can be estimated by dividing the BIP block error ratio by a factor of 1081344.

82.2.16 Descrambler

The descrambler is identical to that used in Clause 49, see 49.2.10 for the definition.

82.2.17 Receive process

The receive process decodes blocks to produce RXD<63:0> and RXC<7:0> for transmission to the XLGMII/CGMII. One XLGMII/CGMII data transfer is decoded from each block. The receive process may insert idle control characters to compensate for the removal of alignment markers. If the PCS receive process spans multiple clock domains, it may also perform clock rate compensation via the deletion of idle control characters or sequence ordered sets or the insertion of idle control characters.

The receive process decodes blocks as specified in the receive state diagram shown in Figure 82–17.

82.2.18 Test-pattern checker

When the receive channel is operating in scrambled idle test-pattern mode, the scrambled idle test-pattern checker checks the bits received via *inst*:IS_UNITDATA_*i*.indication primitives.

The scrambled idle test-pattern checker utilizes the block lock state diagram, the alignment marker state diagram, the PCS deskew state diagram, and the descrambler operating as they do during normal data reception. The BER monitor state diagram is disabled during receive test-pattern mode. When *align_status* is true and the scrambled idle receive test-pattern mode is active, the scrambled idle test-pattern checker observes the sync header and the output from the descrambler. When the sync header and the output of the descrambler is the all idle pattern, a match is detected. When operating in scrambled idle test pattern, the test-pattern error counter counts blocks with a mismatch. Any mismatch indicates an error and shall increment the test-pattern error counter. Because of the error multiplication characteristics of the descrambler, the incoming bit error ratio can be estimated by dividing the 66-bit block error ratio by a factor of 124.

If a Clause 45 MDIO is implemented, then control of the test-pattern reception is from the BASE-R PCS test-pattern control register (bit 3.42.2). In addition errors are counted in the BASE-R PCS test-pattern error counter register (3.43.15:0).

82.2.19 Detailed functions and state diagrams

82.2.19.1 State diagram conventions

The body of this subclause is composed of state diagrams, including the associated definitions of variables, constants, and functions. Should there be a discrepancy between a state diagram and descriptive text, the state diagram prevails.

The notation used in the state diagrams follows the conventions of 21.5. State diagram timers follow the conventions of 14.2.3.2. The notation ++ after a counter or integer variable indicates that its value is to be incremented.

82.2.19.2 State variables

82.2.19.2.1 Constants

EBLOCK_R<71:0>

72 bit vector to be sent to the XLGMII/CGMII containing /E/ in all the eight character locations.

EBLOCK_T<65:0>

66 bit vector to be sent to the PMA containing /E/ in all the eight character locations.

LBLOCK_R<71:0>

72 bit vector to be sent to the XLGMII/CGMII containing one Local Fault ordered set. The Local Fault ordered set is defined in 81.3.4.

LBLOCK_T<65:0>

66 bit vector to be sent to the PMA containing one Local Fault ordered set.

82.2.19.2.2 Variables

`align_status`

A variable set by the PCS deskew process to reflect the status of the PCS lane-to-lane alignment. Set true when all lanes are synchronized and aligned, set false when the deskew process is not complete.

NOTE—If the EEE capability is supported, then this variable is affected by the LPI receive state diagram. If the EEE capability is not supported then this variable is identical to `rx_align_status` controlled according to Figure 82–13.

`alignment_valid`

Boolean variable that is set true if all PCS lanes are aligned. It is valid when each lane is in `am_lock`, with each PCS lane locked to a unique alignment marker from Table 82–2 or Table 82–3, and when all PCS lanes are deskewed. Otherwise, `alignment_valid` is false.

`am_lock<x>`

Boolean variable that is set true when receiver acquires alignment marker delineation for a given lane of the service interface, where $x = 0:3$ for 40GBASE-R and $x = 0:19$ for 100GBASE-R.

`am_slip_done`

Boolean variable that is asserted true when the `AM_SLIP` requested by the alignment marker lock state diagram has been completed indicating that the next candidate 66-bit block position can be tested.

`am_status`

A Boolean variable that is true when all lanes are in `am_lock` and false when at least one lane is not in `am_lock`.

`am_counter_done`

Boolean variable that indicates that the alignment marker counter is done.

`am_valid`

Boolean variable that is set true if received block `rx_coded` is a valid alignment marker. A valid alignment marker matches one of the encodings in Table 82–2, Table 82–3, Table 82–5, or Table 82–6, excluding the `BIP3` and `BIP7` fields.

`ber_test_sh`

Boolean variable that is set true when a new sync header is available for testing and false when `BER_TEST_SH` state is entered. A new sync header is available for testing when the Block Sync process has accumulated enough bits from the PMA to evaluate the header of the next block.

`block_lock<x>`

Boolean variable that is set true when receiver acquires block delineation for a given lane of the service interface, where $x = 0:3$ for 40GBASE-R and $x = 0:19$ for 100GBASE-R.

NOTE—If the EEE capability is supported, then this variable is affected by the LPI receive state diagram. If the EEE capability is not supported, then this variable is identical to `rx_block_lock` controlled according to Figure 82–12.

`current_am`

This variable holds the lane number of the current alignment marker. This is compared to the variable `first_am` to determine if we have alignment marker lock and is always $n \times 16384$ 66-bit blocks away from the `first_am`.

`enable_deskew`

A Boolean variable that indicates the enabling and disabling of the deskew process. Blocks may be discarded whenever deskew is enabled. True when deskew is enabled, false when deskew is disabled.

`first_am`

A variable that holds the lane number of the first alignment marker that is recognized on a given lane. This is used later to compare to future alignment markers.

hi_ber
Boolean variable which is asserted true when the `ber_cnt` equals 97 indicating a bit error ratio $>10^{-4}$.

lane_mapping<x>
This variable indicates which PCS lane is received on lane *x* of the service interface when `am_lock<x> = true`, where *x* = 0:3 for 40GBASE-R and *x* = 0:19 for 100GBASE-R.

PCS_status
A Boolean variable that is true when `align_status` is true and `hi_ber` is false.

r_block_type
This variable contains the `rx_coded<65:0>` vector classification results, returned by the `R_TYPE` function. It can assume one of the following five values {C,S,T,D,E}, as defined by the `R_TYPE` function.

r_block_type_next
This variable contains the `rx_coded<65:0>` vector classification results, returned by the `R_TYPE_NEXT` function. It can assume one of the following five values {C,S,T,D,E}, as defined by the `R_TYPE_NEXT` function.

r_test_mode
Boolean variable that is asserted true when the receiver is in test-pattern mode.

reset
Boolean variable that controls the resetting of the PCS. It is true whenever a reset is necessary including when reset is initiated from the MDIO, during power on, and when the MDIO has put the PCS into low-power mode.

rx_align_status
Variable used by the PCS lane deskew process to reflect the status of the PCS lane-to-lane alignment. This variable is set true when all lanes are synchronized and aligned, set false when the deskew process is not complete.

rx_block_lock<x>
Variable used in Figure 82–12 to reflect the status of the code-group delineation for each lane. This variable is set true when the receiver acquires block delineation.

rx_coded<65:0>
Vector containing the input to the 64B/66B decoder. The format for this vector is shown in Figure 82–5. The leftmost bit in the figure is `rx_coded<0>` and the rightmost bit is `rx_coded<65>`.

rx_down_count
The value that results from the bit-wise exclusive-OR of the Count Down (CD3) byte and the M0 byte of the current received Rapid Alignment Marker (see 82.2.9).

rx_raw<71:0>
Vector containing one XLGMII/CGMII transfer. `RXC<0>` through `RXC<7>` are from `rx_raw<0>` through `rx_raw<7>`, respectively. `RXD<0>` through `RXD<63>` are from `rx_raw<8>` through `rx_raw<71>`, respectively.

sh_valid
Boolean variable that is set true if received block `rx_coded` has valid sync header bits. That is, `sh_valid` is asserted if `rx_coded<0> ≠ rx_coded<1>` and deasserted otherwise.

signal_ok
Boolean variable that is set based on the most recently received value of *inst*:IS_SIGNAL.indication(SIGNAL_OK). It is true if the value was OK and false if the value was FAIL.

slip_done
Boolean variable that is asserted true when the SLIP requested by the Block Lock state diagram has been completed indicating that the next candidate block sync position can be tested.

t_block_type
This variable contains the `tx_raw<71:0>` vector classification results, returned by the `T_TYPE` function. It can assume one of the following five values {C,S,T,D,E}, as defined by the `T_TYPE` function.

test_am

Boolean variable that is set true when a new block is available for testing and false when FIND_1ST state is entered. A new block is available for testing when the Block Sync process has accumulated enough bits from the PMA to evaluate the next block

test_sh

Boolean variable that is set true when a new sync header is available for testing and false when TEST_SH and TEST_SH2 state are entered. A new sync header is available for testing when the Block Sync process has accumulated enough bits from the PMA to evaluate the header of the next block

tx_coded<65:0>

Vector containing the output from the 64B/66B encoder. The format for this vector is shown in Figure 82–5. The leftmost bit in the figure is tx_coded<0> and the rightmost bit is tx_coded<65>.

tx_raw<71:0>

Vector containing one XLGMII/CGMII transfer. TXC<0> through TXC<7> are placed in tx_raw<0> through tx_raw<7>, respectively. TXD<0> through TXD<63> are placed in tx_raw<8> through tx_raw<71>, respectively.

The following variables are used only for the EEE capability.

down_count_done

Boolean variable that indicates that the down_count counter has reached zero.

down_count_enable

Boolean variable controlling decrement of the counter down_count. This variable is set by the LPI transmit state diagram.

energy_detect

A parameter generated by the PMA/PMD sublayer to reflect the state of the received signal. In the PMD this has the same definition as parameter signal_detect and is passed through without modification by the PMA (and FEC).

first_rx_lpi_active

Boolean variable first_rx_lpi_active is set true when the receiver is in state RX_ACTIVE in the LPI receive state diagram (see Figure 82–19) and R_TYPE(rx_coded) = LI and LPI_FW = FALSE and is otherwise false.

LPI_FW

Boolean variable controlling the wake mode for the LPI transmit and receive functions. This variable is set true when the link is to use the fast wake mechanism, and false when the link is to use the optional deep sleep mechanism for each direction. This variable defaults true and may only be set to false if the optional deep sleep mode is supported.

rx_lpi_active

A Boolean variable that is set to true when the receiver is in a low power state and set to false when it is in an active state and capable of receiving data.

Rx LPI indication:

A Boolean variable indicating the current state of the receive LPI function. This flag is set to true (register bit set to one) when the LPI receive state diagram is in any state other than RX_ACTIVE. This status is reflected in MDIO register 3.1.8. A latch high view of this status is reflected in MDIO register 3.1.10 (Rx LPI received).

rx_mode

A variable reflecting state of the LPI receive function as defined in Figure 82–19. The parameter has one of two values: DATA and QUIET.

scrambler_bypass

This Boolean variable indicates whether the Tx PCS scrambler is to be bypassed in order to assist rapid synchronization following low power idle. When set to TRUE, the PCS passes the unscrambled data from the scrambler input rather than the scrambled data from the scrambler output. The scrambler continues to operate normally, shifting input data into the delay line. When scrambler_bypass is set to FALSE the PCS passes scrambled data from the scrambler output.

scr_bypass_enable

A Boolean variable used to indicate to the transmit LPI state diagram that the scrambler bypass option is required. The PHY shall set scr_bypass_enable = TRUE if Clause 74 FEC is in use. The PHY shall set scr_bypass_enable = FALSE if this FEC is not in use.

tx_mode

A variable reflecting state of the LPI transmit function as defined in Figure 82–18. When tx_mode is set to QUIET the sublayer may go into a low power state.

82.2.19.2.3 Functions

AM_SLIP

Causes the next candidate block position to be tested. The precise method for determining the next candidate block position is not specified and is implementation dependent. However, an implementation shall ensure that all possible blocks are evaluated.

DECODE(rx_coded<65:0>)

Decodes the 66-bit vector returning rx_raw<71:0>, which is sent to the XLGMII/CGMII. The DECODE function shall decode the block as specified in 82.2.3.

ENCODE(tx_raw<71:0>)

Encodes the 72-bit vector returning tx_coded<65:0> of which tx_coded<65:2> is sent to the scrambler. The two bits of the sync header bypass the scrambler. The ENCODE function shall encode the block as specified in 82.2.3.

R_TYPE(rx_coded<65:0>)

This function classifies the current rx_coded<65:0> vector as belonging to one of the following types, depending on its contents. The classification results are returned via the r_block_type variable.

Values: C; The vector contains a sync header of 10 and one of the following:

- a) A block type field of 0x1E and eight valid control characters other than /E/ or /LI/;
- b) A block type field of 0x4B.

LI; For EEE capability, the LI type is supported where the vector contains a sync header of 10, a block type field of 0x1E and eight control characters of 0x06 (/LI/).

S; The vector contains a sync header of 10 and the following:

- a) A block type field of 0x78.

T; The vector contains a sync header of 10, a block type field of 0x87, 0x99, 0xAA, 0xB4, 0xCC, 0xD2, 0xE1 or 0xFF and all control characters are valid.

D; The vector contains a sync header of 01.

E; The vector does not meet the criteria for any other value.

Valid control characters are specified in Table 82–1.

NOTE—A PCS that does not support EEE classifies vectors containing one or more /LI/ control characters as type E.

R_TYPE_NEXT

This function classifies the 66-bit rx_coded vector that immediately follows the current rx_coded<65:0> vector as belonging to one of the five types defined in R_TYPE, depending on its contents. It is intended to perform a prescient end of packet check. The classification results are returned via the r_block_type_next variable.

SLIP

Causes the next candidate block sync position to be tested. The precise method for determining the next candidate block sync position is not specified and is implementation dependent. However, an implementation shall ensure that all possible bit positions are evaluated.

T_TYPE = (tx_raw<71:0>)

This function classifies each 72-bit tx_raw vector as belonging to one of the following types depending on its contents. The classification results are returned via the t_block_type variable.

Values: C; The vector contains one of the following:

- a) Eight valid control characters other than /O/, /S/, /T/, /LI/, and /E/;
- b) One valid ordered set.

LI; For EEE capability, this vector contains eight /LI/ characters.

S; The vector contains an /S/ in its first character, and all characters following the /S/ are data characters.

T; The vector contains a /T/ in one of its characters, all characters before the /T/ are data characters, and all characters following the /T/ are valid control characters other than /O/, /S/ and /T/.

D; The vector contains eight data characters.

E; The vector does not meet the criteria for any other value.

A tx_raw character is a control character if its associated TxC bit is asserted. A valid control character is one containing an XLGMII/CGMII control code specified in Table 82–1. A valid ordered set consists of a valid /O/ character in the first character and data characters in the seven characters following the /O/. A valid /O/ is any character with a value for O code in Table 82–1.

NOTE—A PCS that does not support EEE classifies vectors containing one or more /LI/ control characters as type E.

82.2.19.2.4 Counters

am_counter

This counter counts 66-bit blocks that separate two consecutive alignment markers. If the optional EEE deep sleep capability is supported, when rx_lpi_active is TRUE and LPI_FW is FALSE, the terminal count is 7 for 100GBASE-R PCS and 15 for 40GBASE-R PCS. If the optional EEE deep sleep capability is not supported, or rx_lpi_active is FALSE, the terminal count is 16383.

am_invld_cnt

Count of the number of invalid alignment markers seen in a row. This counter is always reset to all zeros when a valid marker is detected.

ber_count

A 22-bit counter that counts each time BER_BAD_SH state is entered. This counter is reflected in MDIO register bits 3.33.13:8 and 3.44.15:0.

ber_cnt

Count up to a maximum of 97 of the number of invalid sync headers within the current 1.25 ms (40GBASE-R) or 500 μs (100GBASE-R) period.

errored_block_count:

When the receiver is in normal mode, this 22-bit counter counts once for each time RX_E state is entered. This counter is reflected in MDIO register bits 3.33.7:0 and 3.45.13:0.

sh_cnt

Count of the number of sync headers checked within the current 64 or 1024 block window.

sh_invld_cnt

Count of the number of invalid sync headers within the current 64 or 1024 block window.

test_pattern_error_count:

When the receiver is in test-pattern mode, this 16-bit counter counts errors as described in 82.2.18. This counter is reflected in MDIO register bits 3.43.15:0.

The following counters are used only for the EEE capability:

down_count

A counter that is used in rapid alignment markers and is decremented each time a RAM is sent while variable down_count_enable = true. The counter initial value is set by the LPI transmit state diagram. When the down_count counter reaches zero, it sets the variable down_count_done = true.

wake_error_counter

A counter that is incremented each time that the LPI receive state diagram enters the RX_WTF state indicating that a wake time fault has been detected. The counter is reflected in register 3.22 (see 45.2.3.12).

82.2.19.2.5 Timers

State diagram timers follow the conventions of 14.2.3.2.

xus_timer

Timer that is triggered every 1.25 ms +1%, –25% for 40GBASE-R or 500 μ s +1%, –25% for 100GBASE-R.

The following timers are used only for the EEE capability.

one_us_timer

A timer used to count approximately 1 μ s intervals. The timer terminal count is set to T_{IU} . When the timer reaches terminal count, it sets one_us_timer_done = true.

rx_tq_timer

This timer is started when the PCS receiver enters the RX_SLEEP state. The timer terminal count is set to T_{QR} . When the timer reaches terminal count, it sets rx_tq_timer_done = true.

rx_tw_timer

This timer is started when the PCS receiver enters the RX_WAKE state. The timer terminal count is set to a value no larger than the maximum value given for T_{WR} in Table 82–9. When the timer reaches terminal count, it sets rx_tw_timer_done = true.

rx_wf_timer

This timer is started when the PCS receiver enters the RX_WTF state, indicating that the receiver has encountered a wake time fault. The rx_wf_timer allows the receiver an additional period in which to synchronize or return to the QUIET state before a link failure is indicated. The timer terminal count is set to T_{WTF} . When the timer reaches terminal count, it sets the rx_wf_timer_done = true.

scr_byp_timer

This timer is started when the PCS transmitter enters the TX_SCR_BYPASS state. The timer terminal count is set to T_{BYP} . When the timer reaches terminal count, it sets the scr_byp_timer_done = true.

tx_ts_timer

This timer is started when the PCS transmitter enters the TX_SLEEP state. The timer terminal count is set to T_{SL} . When the timer reaches terminal count, it sets the tx_ts_timer_done = true.

tx_tq_timer

This timer is started when the PCS transmitter enters the TX_QUIET state. The timer terminal count is set to T_{QL} . When the timer reaches terminal count, it sets the tx_tq_timer_done = true.

tx_tw_timer

This timer is started when the PCS transmitter enters the TX_WAKE state. The timer terminal count is set to T_{WL} . When the timer reaches terminal count, it sets the tx_tw_timer_done = true.

tx_tw2_timer

This timer is started when the PCS transmitter enters the TX_WAKE2 state. The timer terminal count is set to T_{WL2} . When the timer reaches terminal count, it sets the tx_tw2_timer_done = true.

82.2.19.3 State diagrams

The 40GBASE-R PCS shall implement four block lock processes as depicted in Figure 82–12. The 100GBASE-R PCS shall implement twenty block lock processes as depicted in Figure 82–12. A block lock process operates independently on each lane. Each block lock process looks for 64 valid sync headers in a row to declare lock. A valid sync header is either a 01 or a 10. Once in lock, the lock process looks for 65 invalid sync headers within a 1024 sync window to declare out of lock. An invalid sync header is a 11 or 00. Once block lock is achieved on a lane, then the alignment marker process starts.

The 40GBASE-R PCS shall implement four alignment marker lock processes as depicted in Figure 82–13. The 100GBASE-R PCS shall implement twenty alignment marker lock processes as depicted in Figure 82–13. An alignment marker lock process operates independently on each lane. The alignment marker lock state diagram shown in Figure 82–13 determines when the PCS has obtained alignment marker lock to the received bit stream for a given lane of the service interface. Each alignment marker lock process looks for two valid alignment markers 16384 66-bit blocks apart to gain alignment marker lock. On a given lane of the service interface, lane markers have to match each other and an entry from Table 82–2 for 100GBASE-R or Table 82–3 for 40GBASE-R. Note that the BIP₃ and BIP₇ fields are excluded from the markers when making a match to each other or the tables. Once in lock, a lane will go out of alignment marker lock if four markers are received in a row that do not match the alignment marker that the lane is currently locked to. When the alignment marker lock process achieves lock for a lane, and if a Clause 45 MDIO is implemented, the PCS shall record the number of the PCS lane received on that lane of the service interface in the appropriate lane mapping register (3.400 to 3.419).

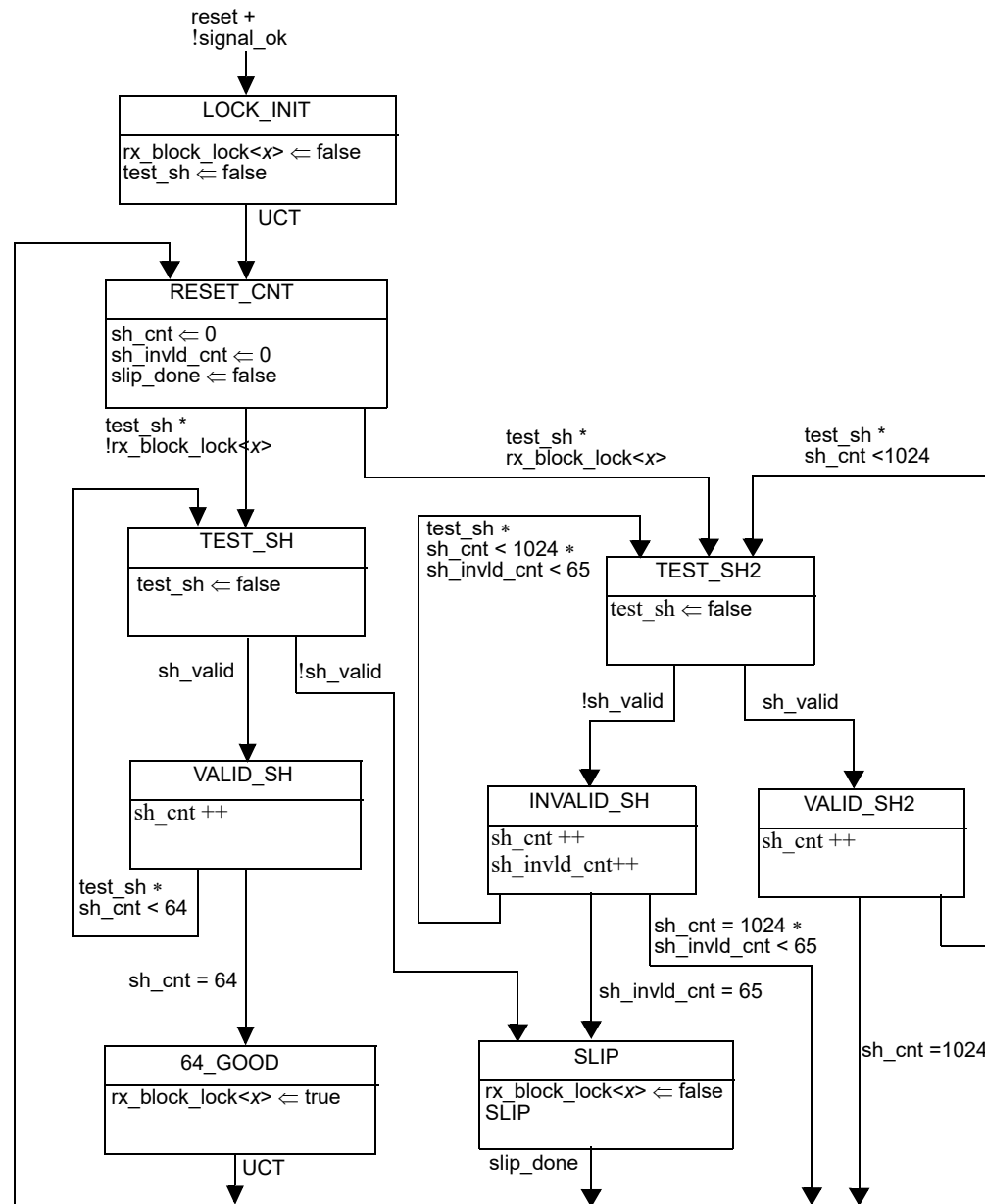
The PCS shall run the deskew process as depicted in Figure 82–14. The PCS deskew process is responsible for determining if the PCS is capable of presenting coherent data to the XLGMII/CGMII. The deskew process ensures that all PCS lanes have alignment marker lock, are locked to different alignment markers, and that the Skew is within the boundaries of what the PCS can deskew.

The BER Monitor state diagram shown in Figure 82–15 monitors the received aggregate signal for high bit error ratio. The high BER state shall be entered if 97 invalid 66-bit sync headers are detected within a 500 μ s window for 100GBASE-R, or a 1.25 ms window for 40GBASE-R. The high BER state is exited once there are less than 97 invalid sync headers in the same window.

The Transmit state diagram shown in Figure 82–16 controls the encoding of transmitted blocks. It makes exactly one transition for each transmit block processed. Though the Transmit state diagram sends Local Fault ordered sets when reset is asserted, the scrambler may not be operational during reset. Thus, the Local Fault ordered sets may not appear on the PMA/FEC service interface.

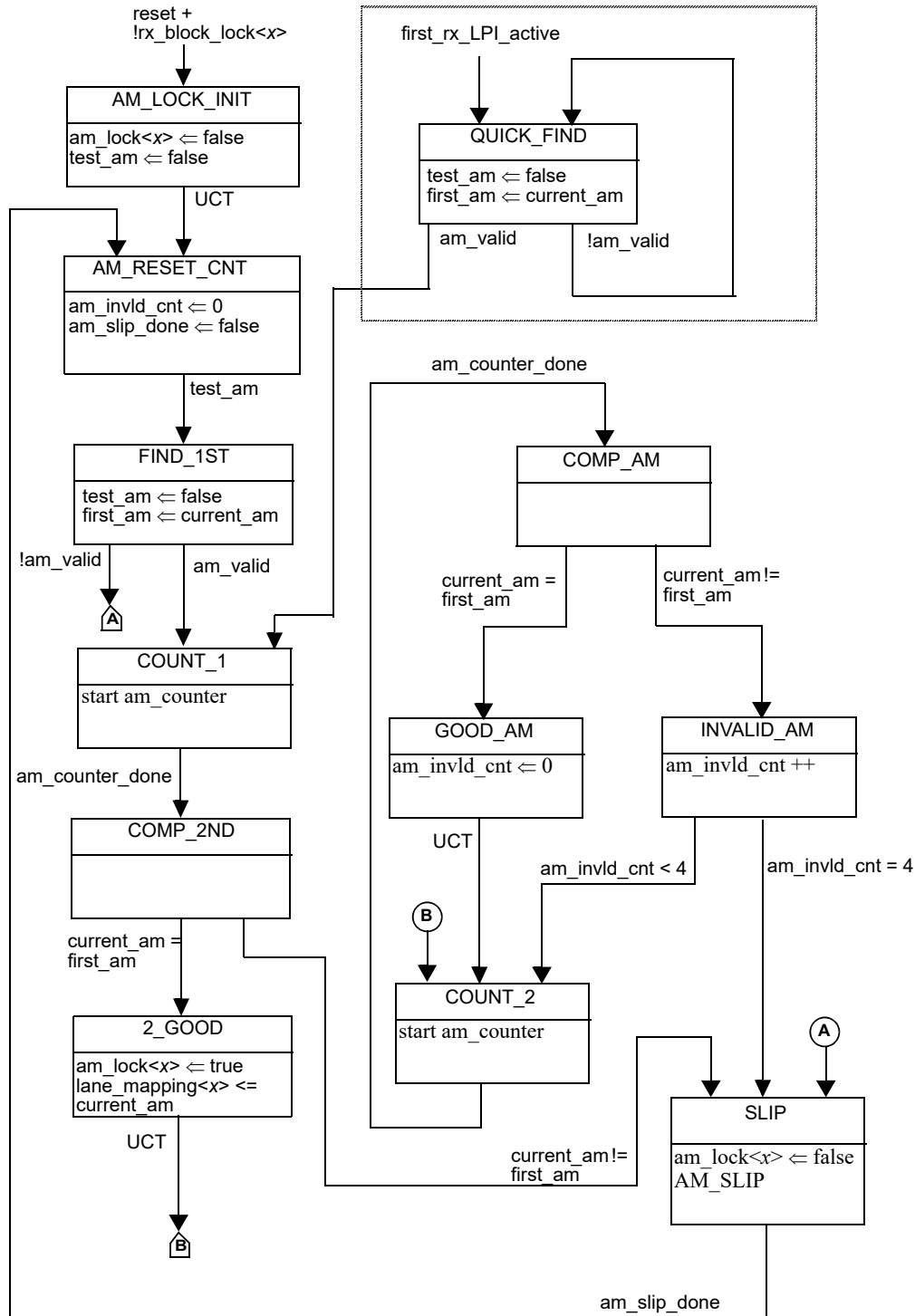
The Receive state diagram shown in Figure 82–17 controls the decoding of received blocks. It makes exactly one transition for each receive block processed.

The PCS shall perform the functions of block lock, alignment marker lock, PCS deskew, BER Monitor, Transmit, and Receive as specified in the respective state diagrams.



NOTE— rx_block_lock<x> refers to the received lane x of the service interface, where x = 0:3 (for 40GBASE-R) or 0:19 (for 100GBASE-R)

Figure 82-12—Block lock state diagram



NOTE 1— am_lock<x> refers to the received lane x of the service interface, where x = 0:3 (for 40GBASE-R) or 0:19 (for 100GBASE-R)

NOTE 2—Optional state (inside the dotted box) is only required to support EEE capability.

Figure 82-13—Alignment marker lock state diagram

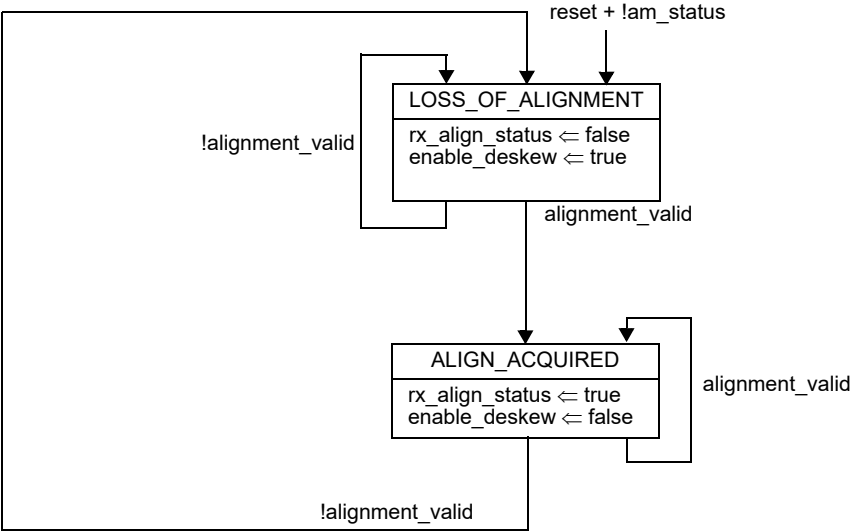


Figure 82–14—PCS deskew state diagram

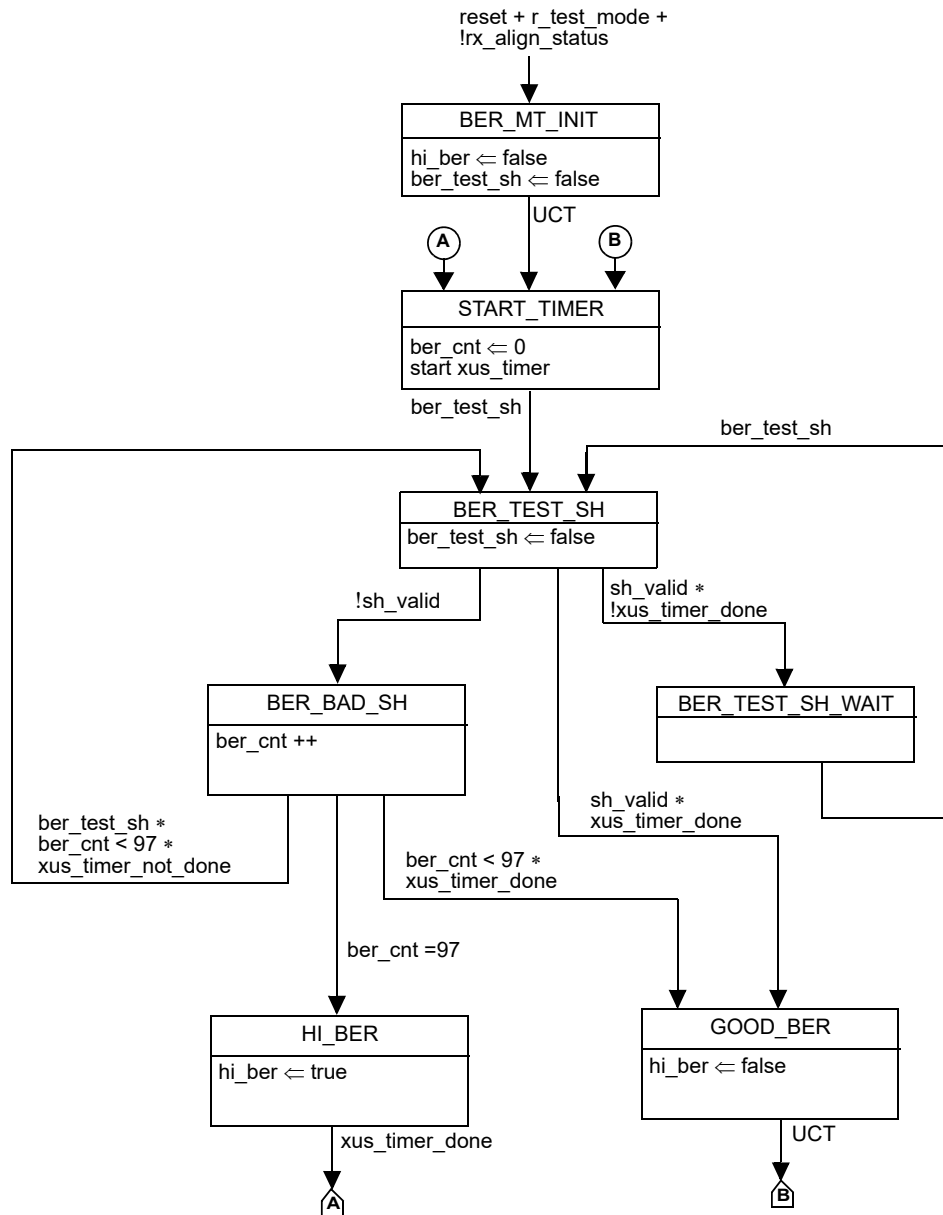


Figure 82–15—BER monitor state diagram

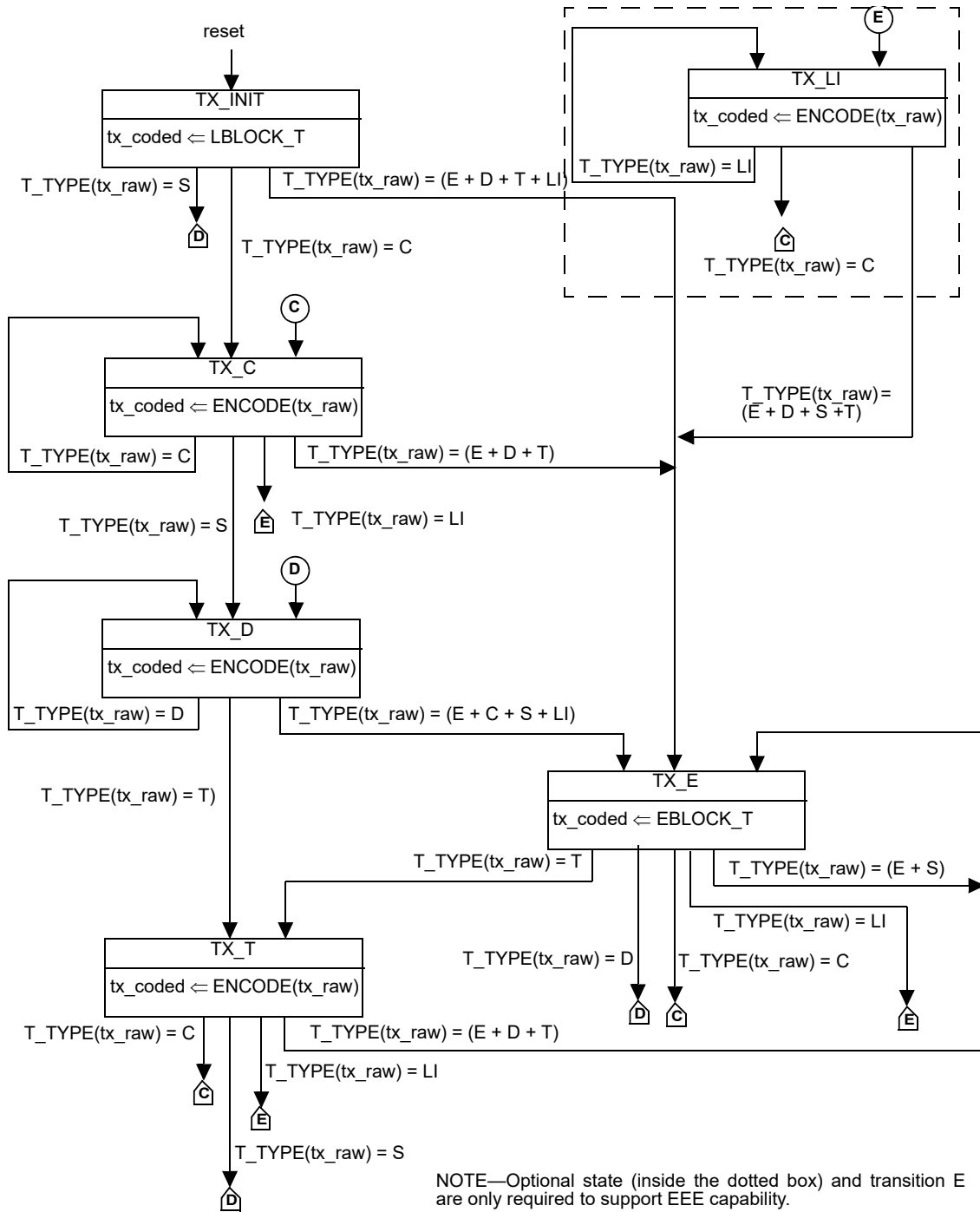
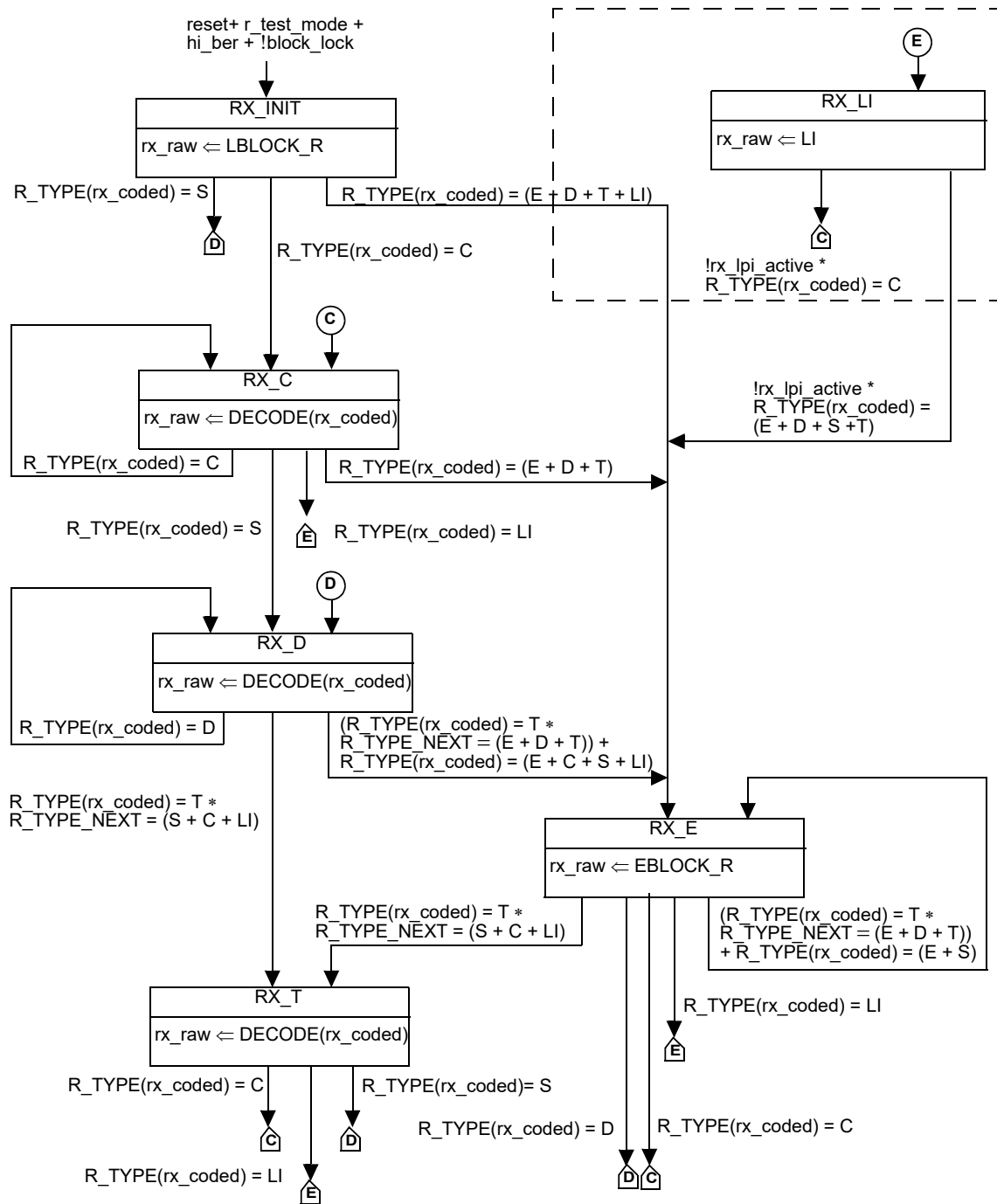


Figure 82–16—Transmit state diagram



NOTE—Optional state (inside the dotted box) and transition E are only required to support IEEE capability.

Figure 82–17—Receive state diagram

82.2.19.3.1 LPI state diagrams

A PCS that supports the EEE capability shall implement the LPI transmit and receive processes as shown in Figure 82–18 and Figure 82–19. The transmit LPI state diagram controls tx_mode, which disables the transmitter when it is set to QUIET. The receive LPI state diagram controls block_lock during LPI and signals the end of LPI to the receive state diagram.

Following a period of LPI, the receiver is required to achieve block synchronization within the wake-up time specified (see Figure 82–19). The implementation of the block synchronization state diagram should use techniques to ensure that block lock is achieved with minimal numbers of slip attempts. If fast wake is selected then the receiver is expected to maintain sufficient state to allow much faster wake up.

The LPI functions shall use timer values for these state diagrams as shown in Table 82–8 for transmit and Table 82–9 for receive.

Table 82–8—Transmitter LPI timing parameters

Parameter	Description	Min	Max	Units
T _{SL}	Local Sleep Time from entering the TX_SLEEP state to when tx_mode is set to QUIET.	0.9	1.1	μs
T _{QL}	Local Quiet Time from when tx_mode is set to QUIET to entry into the TX_WAKE state.	1.7	1.8	ms
T _{WL}	Time spent in the TX_WAKE state.	1.5	1.6	μs
T _{WL2}	Time spent in the TX_WAKE2 state.	2.4	2.5	μs
T _{BYP}	Time spent in the TX_SCR_BYPASS state, 40 Gb/s operation.	0.9	1.1	μs
T _{BYP}	Time spent in the TX_SCR_BYPASS state, 100 Gb/s operation.	1.9	2.1	μs
T _{IU}	Time spent in the TX_ALERT state.	1.15	1.3	μs

Table 82–9—Receiver LPI timing parameters

Parameter	Description	Min	Max	Units
T _{QR}	Time the receiver waits for energy_detect to be set to true while in the RX_QUIET state before asserting receive fault.	2	3	ms
T _{WR}	Time the receiver waits in the RX_WAKE state before indicating a wake time fault, scr_bypass_enable = FALSE.	—	4.5	μs
T _{WR}	Time the receiver waits in the RX_WAKE state before indicating a wake time fault, scr_bypass_enable = TRUE, 40 Gb/s.	—	5.5	μs
T _{WR}	Time the receiver waits in the RX_WAKE state before indicating a wake time fault, scr_bypass_enable = TRUE, 100 Gb/s.	—	6.5	μs
T _{WTF}	Wake time fault recovery time.	—	10	ms

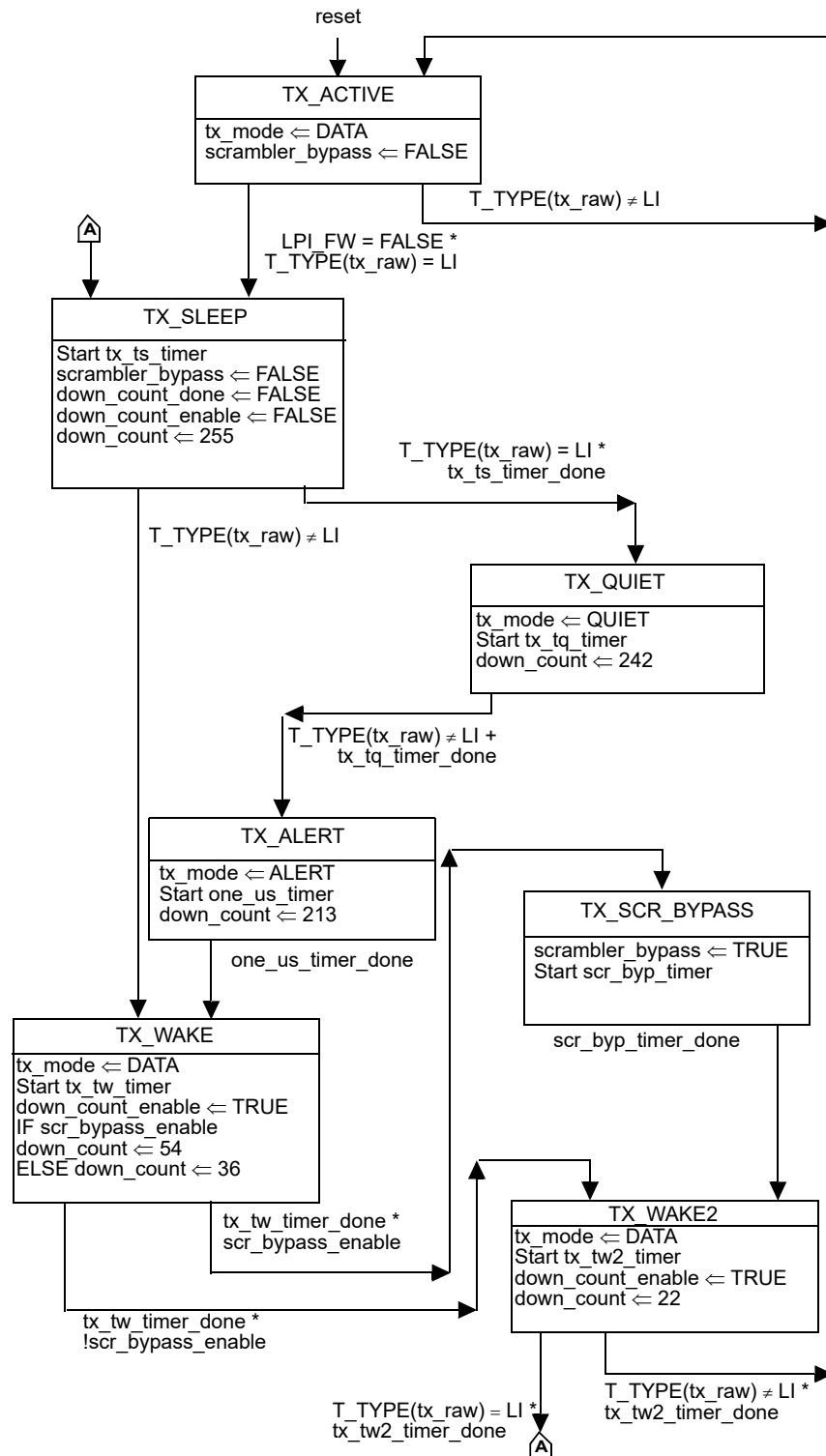


Figure 82–18—LPI Transmit state diagram

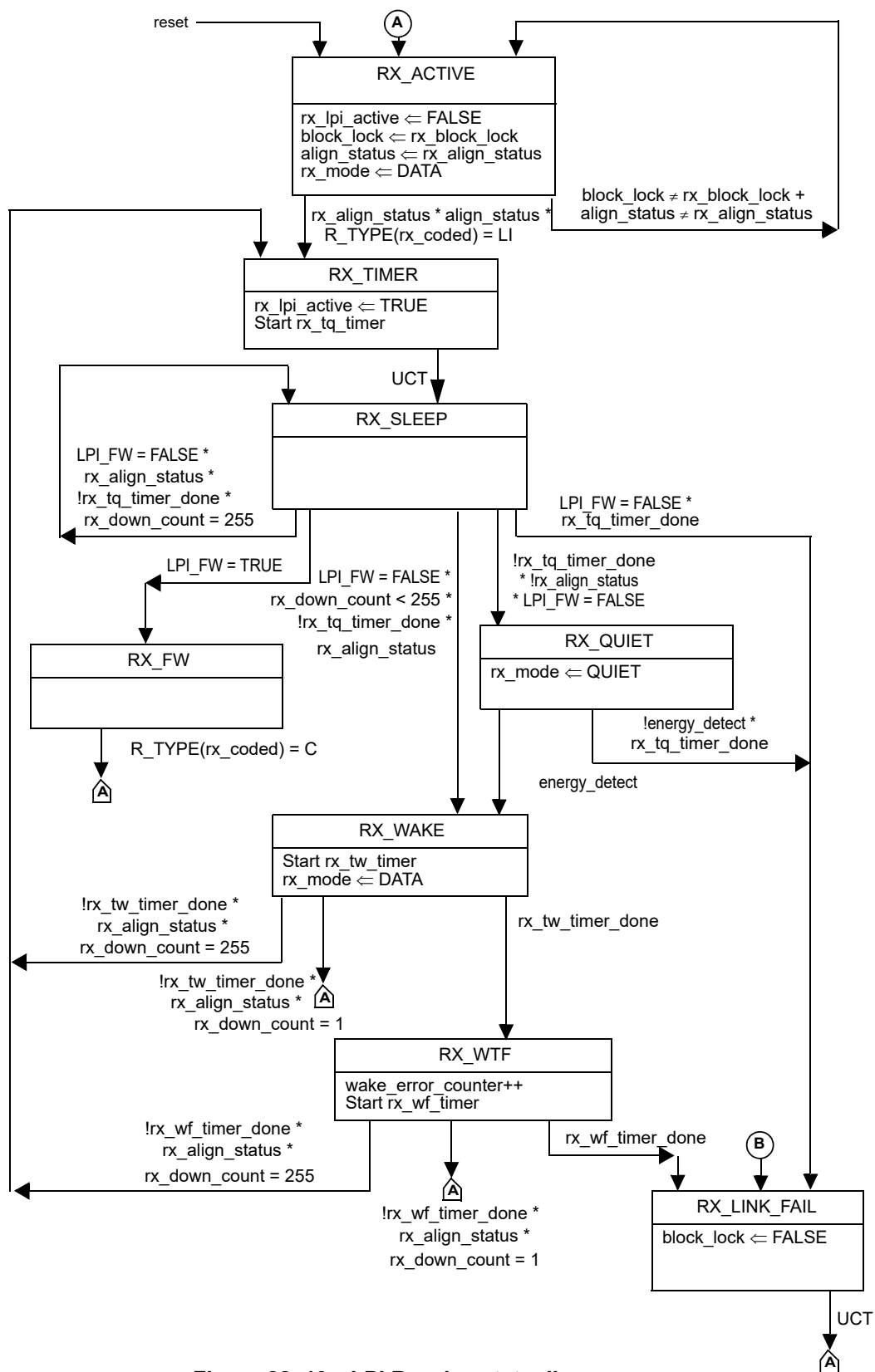


Figure 82-19—LPI Receive state diagram

82.3 PCS Management

The following objects apply to PCS management. If an MDIO Interface is provided (see Clause 45), they are accessed via that interface. If not, it is recommended that an equivalent access be provided.

82.3.1 PCS MDIO function mapping

The optional MDIO capability described in Clause 45 defines several variables that may provide control and status information for and about the PCS. Mapping of MDIO control variables to PCS control variables is shown in Table 82–10. Mapping of MDIO status variables to PCS status variables is shown in Table 82–11.

Table 82–10—MDIO/PCS control variable mapping

MDIO control variable	PCS register name	Register/ bit number	PCS control variable
Reset	PCS control 1 register	3.0.15	reset
Loopback	PCS control 1 register	3.0.14	Loopback
Transmit test-pattern enable	BASE-R PCS test-pattern control register	3.42.3	tx_test_mode
Receive test-pattern enable	BASE-R PCS test-pattern control register	3.42.2	rx_test_mode
LPI_FW	LPI fast wake enable	3.20.0	LPI_FW

Table 82–11—MDIO/PCS status variable mapping

MDIO status variable	PCS register name	Register/ bit number	PCS status variable
BASE-R and 10GBASE-T receive link status	BASE-R and 10GBASE-T PCS status 1 register	3.32.12	PCS_status
BASE-R and 10GBASE-T PCS high BER	BASE-R and 10GBASE-T PCS status 1 register	3.32.1	hi_ber
Block lock x	Multi-lane BASE-R PCS alignment status 1 and 2 registers	3.50.7:0 3.51.11:0	block_lock< x >
Lane x aligned	Multi-lane BASE-R PCS alignment status 3 and 4 registers	3.52.7:0 3.53.11:0	am_lock< x >
PCS lane alignment status	Multi-lane BASE-R PCS alignment status 1 register	3.50.12	align_status
BER	BASE-R and 10GBASE-T PCS status 2 register BER high order counter register	3.33.13:8 3.44.15:0	ber_count
Errored blocks	BASE-R and 10GBASE-T PCS status 2 register Errored blocks high order counter register	3.33.7:0 3.45.13:0	errored_block_count

Table 82–11—MDIO/PCS status variable mapping (*continued*)

MDIO status variable	PCS register name	Register/ bit number	PCS status variable
Test-pattern error counter	BASE-R PCS test-pattern error counter register	3.43.15:0	test_pattern_error_count
BIP error counter, lane <i>x</i>	BIP error counter, lane <i>x</i> register	3.200 through 3.219	bip_counter
Lane <i>x</i> mapping	Lane <i>x</i> mapping register	3.400 through 3.419	lane_mapping
Tx LPI indication	Tx LPI indication	3.1.9	Tx LPI indication
Tx LPI received	Tx LPI received	3.1.11	Tx LPI received
Rx LPI indication	Rx LPI indication	3.1.8	Rx LPI indication
Rx LPI received	Rx LPI received	3.1.10	Rx LPI received
Wake_error_counter	Wake_error_counter	3.22	Wake_error_counter

82.4 Loopback

If a Clause 45 MDIO is implemented, then the PCS shall be placed in the loopback mode when the loopback bit from the PCS control 1 register (bit 3.0.14) is set to a one. In this mode, the PCS shall accept data on the transmit path from the XLGMII/CGMII and return it on the receive path to the XLGMII/CGMII. In addition, the PCS shall transmit what it receives from the XLGMII/CGMII to the PMA/FEC sublayer, and shall ignore all data presented to it by the PMA/FEC sublayer.

82.5 Delay constraints

The maximum delay contributed by the 40GBASE-R PCS (sum of transmit and receive delays at one end of the link) shall be no more than 11264 BT (22 pause_quanta or 281.6 ns). The maximum delay contributed by the 100GBASE-R PCS (sum of transmit and receive delays at one end of the link) shall be no more than 35328 BT (69 pause_quanta or 353.28 ns). A description of overall system delay constraints and the definitions for bit times and pause_quanta can be found in 80.4 and its references.

82.6 Auto-Negotiation

The following requirements apply to a PCS used with a 40GBASE-KR4 PMD, 40GBASE-CR4 PMD, 100GBASE-CR10, 100GBASE-CR4, 100GBASE-CR2, 100GBASE-KR4, 100GBASE-KP4, or 100GBASE-KR2 PMD where support for the Auto-Negotiation process defined in Clause 73 is mandatory. The PCS shall support the primitive AN_LINK.indication(link_status) (see 73.9). The parameter link_status shall take the value FAIL when PCS_status=false and the value OK when PCS_status=true. The primitive shall be generated when the value of link_status changes.

82.7 Protocol implementation conformance statement (PICS) proforma for Clause 82, Physical Coding Sublayer (PCS) for 64B/66B, type 40GBASE-R and 100GBASE-R¹⁴⁷

82.7.1 Introduction

The supplier of a protocol implementation that is claimed to conform to Clause 82, Physical Coding Sublayer (PCS) for 64B/66B, type 40GBASE-R and 100GBASE-R, shall complete the following protocol implementation conformance statement (PICS) proforma. A detailed description of the symbols used in the PICS proforma, along with instructions for completing the PICS proforma, can be found in Clause 21.

82.7.2 Identification

82.7.2.1 Implementation identification

Supplier ¹	
Contact point for inquiries about the PICS ¹	
Implementation Name(s) and Version(s) ^{1,3}	
Other information necessary for full identification—e.g., name(s) and version(s) for machines and/or operating systems; System Name(s) ²	
NOTE 1— Required for all implementations. NOTE 2— May be completed as appropriate in meeting the requirements for the identification. NOTE 3—The terms Name and Version should be interpreted appropriately to correspond with a supplier's terminology (e.g., Type, Series, Model).	

82.7.2.2 Protocol summary

Identification of protocol standard	IEEE Std 802.3-2022, Clause 82, Physical Coding Sublayer (PCS) for 64B/66B, type 40GBASE-R and 100GBASE-R
Identification of amendments and corrigenda to this PICS proforma that have been completed as part of this PICS	
Have any Exception items been required? No ☐ Yes ☐ (See Clause 21; the answer Yes means that the implementation does not conform to IEEE Std 802.3-2022.)	

Date of Statement	
-------------------	--

¹⁴⁷Copyright release for PICS proformas: Users of this standard may freely reproduce the PICS proforma in this subclause so that it can be used for its intended purpose and may further publish the completed PICS.

82.7.3 Major capabilities/options

Item	Feature	Subclause	Value/Comment	Status	Support
*PCS40	PCS for 40GBASE-R	82.1.1		O	Yes [] No []
*PCS100	PCS for 100GBASE-R	82.1.1		O	Yes [] No []
XGE40	XLGMII logical interface	81, 82.1.4	Logical interface is supported	O	Yes [] No []
XGE100	CGMII logical interface	81, 82.1.4	Logical interface is supported	O	Yes [] No []
*MD	MDIO	45, 82.3	Registers and interface supported	O	Yes [] No []
PMA	Supports operation directly connected to a PMA	82.1.4.2		O.1	Yes [] No []
FEC	Supports operation directly connected to an FEC sublayer	82.1.4.2		O.1	Yes [] No []
*JTM	Supports test-pattern mode	82.2.1		PMA:M	Yes [] No [] N/A []
*LPI	Implementation of LPI	82.2.3.4		O	Yes [] No []

82.7.4 PICS proforma tables for PCS, type 40GBASE-R and 100GBASE-R

82.7.4.1 Coding rules

Item	Feature	Subclause	Value/Comment	Status	Support
C1	Encoder (and ENCODE function) implements the code as specified	82.2.3 and 82.2.19.2.3		M	Yes []
C2	Decoder (and DECODE function) implements the code as specified	82.2.3 and 82.2.19.2.3		M	Yes []
C3	Only valid block types are transmitted	82.2.3.3		M	Yes []
C4	Invalid block types are treated as an error	82.2.3.3		M	Yes []
C5	Only valid control characters are transmitted	82.2.3.4		M	Yes []
C6	Invalid control characters are treated as an error	82.2.3.4		M	Yes []
C7	Idles do not interrupt data	82.2.3.6		M	Yes []
C8	IDLE control code insertion and deletion	82.2.3.6	Insertion or Deletion in groups of 8 /I/s	M	Yes []
C9	Sequence ordered set deletion	82.2.3.9	Only one whole ordered set of two consecutive sequence ordered sets may be deleted	M	Yes []

82.7.4.2 Scrambler and Descrambler

Item	Feature	Subclause	Value/Comment	Status	Support
S1	Scrambler	82.2.5	Performs as shown in Figure 49–8	M	Yes []
S2	Descrambler	82.2.16	Performs as shown in Figure 49–10	M	Yes []

82.7.4.3 Deskew and Reordering

Item	Feature	Subclause	Value/Comment	Status	Support
DR1	Deskew	82.2.13	Able to deskew up to the value in Table 82–7	M	Yes []
DR2	Reordering	82.2.14	Performs reordering	M	Yes []

82.7.4.4 Alignment Markers

Item	Feature	Subclause	Value/Comment	Status	Support
AM1	Alignment marker insertion	82.2.7	Alignment markers are inserted periodically as described in 82.2.7	M	Yes []
AM2	Alignment marker form	82.2.7	Alignment markers are formed as described in 82.2.7	M	Yes []
AM3	Alignment marker removal	82.2.15	Alignment markers are removed as described in 82.2.15	M	Yes []
AM4	Lane mapping	82.2.19.3	PCS lane number is captured	MD:M	Yes [] N/A []

82.7.4.5 Test-pattern modes

Item	Feature	Subclause	Value/Comment	Status	Support
JT1	Scrambled idle transmit test-pattern generator is implemented	82.2.11	Performs as in 82.2.11	JTM:M	Yes [] N/A []
JT2	Scrambled idle receive test-pattern checker is implemented	82.2.18	Performs as in 82.2.18	JTM:M	Yes [] N/A []
JT3	Transmit and receive test-pattern modes can operate simultaneously	82.2.1		JTM:M	Yes [] N/A []

82.7.4.6 Bit order

Item	Feature	Subclause	Value/Comment	Status	Support
B1	Transmit bit order	82.2.3.2	Placement of bits into the PCS lanes as shown in Figure 82–3	M	Yes []
B2	Receive bit order	82.2.3.2	Placement of bits into the XLGMII/CGMII as shown in Figure 82–4	M	Yes []

82.7.4.7 Management

Item	Feature	Subclause	Value/Comment	Status	Support
M1	Equivalent access to PCS Management objects is provided	82.3		!MD:O	Yes [] No [] N/A []

82.7.4.8 State diagrams

Item	Feature	Subclause	Value/Comment	Status	Support
SM1	40GBASE-R Block Lock	82.2.19.3	Implements 4 block lock processes as depicted in Figure 82–12	PCS40:M	Yes [] N/A []
SM2	100GBASE-R Block Lock	82.2.19.3	Implements 20 block lock processes as depicted in Figure 82–12	PCS100:M	Yes [] N/A []
SM3	The SLIP function evaluates all possible bit positions	82.2.19.2.3		M	Yes []
SM4	40GBASE-R Alignment Marker Lock	82.2.19.3	Implements 4 alignment marker lock processes as depicted in Figure 82–13	PCS40:M	Yes [] N/A []
SM5	100GBASE-R Alignment Marker Lock	82.2.19.3	Implements 20 alignment marker lock processes as depicted in Figure 82–13	PCS100:M	Yes [] N/A []
SM6	The AM_SLIP functions evaluates all possible blocks	82.2.19.2.3		M	Yes []
SM7	40GBASE-R PCS deskew state diagram	82.2.19.3	Meets the requirements of Figure 82–14	PCS40:M	Yes [] N/A []
SM8	100GBASE-R PCS deskew state diagram	82.2.19.3	Meets the requirements of Figure 82–14	PCS100:M	Yes [] N/A []
SM9	40GBASE-R BER Monitor	82.2.19.3	Meets the requirements of Figure 82–15 with xus_timer_done set to 1.25 ms	PCS40:M	Yes [] N/A []
SM10	100GBASE-R BER Monitor	82.2.19.3	Meets the requirements of Figure 82–15 with xus_timer_done set to 500 μ s	PCS100:M	Yes [] N/A []
SM11	40GBASE-R Transmit process	82.2.19.3	Meets the requirements of Figure 82–16	PCS40:M	Yes [] N/A []
SM12	100GBASE-R Transmit process	82.2.19.3	Meets the requirements of Figure 82–16	PCS100:M	Yes [] N/A []
SM13	40GBASE-R and 100GBASE-R Receive process	82.2.19.3	Meets the requirements of Figure 82–17	PCS40:M	Yes [] N/A []
SM14	100GBASE-R Receive process	82.2.19.3	Meets the requirements of Figure 82–17	PCS100:M	Yes [] N/A []

82.7.4.9 Loopback

Item	Feature	Subclause	Value/Comment	Status	Support
L1	Supports loopback	82.4	Performs as in 82.4	M	Yes []
L2	When in loopback, transmits what it receives from the XLGMII/CGMII	82.4	Performs as in 82.4	M	Yes []

82.7.4.10 Delay constraints

Item	Feature	Subclause	Value/Comment	Status	Support
TIM1	PCS Delay Constraint	82.5	No more than 11264 BT for sum of transmit and receive path delays for 40GBASE-R and 35328 BT for 100GBASE-R	M	Yes []

82.7.4.11 Auto-Negotiation for Backplane Ethernet functions

Item	Feature	Subclause	Value/Comment	Status	Support
*AN1	Support for use with a 40GBASE-KR4, 40GBASE-CR4, 100GBASE-CR10, 100GBASE-CR4, 100GBASE-CR2, 100GBASE-KR4, 100GBASE-KP4, or 100GBASE-KR2 PMD	82.6	AN technology dependent interface described in Clause 73	O	Yes [] No []
AN2	AN_LINK.indication primitive	82.6	Support of the primitive AN_LINK.indication(link_status)	AN1:M	Yes [] N/A []
AN3	link_status parameter	82.6	Takes the value OK or FAIL, as described in 82.6	AN1:M	Yes [] N/A []
AN4	Generation of AN_LINK.indication primitive	82.6	Generated when the value of link_status changes	AN1:M	Yes [] N/A []

82.7.4.12 LPI functions

Item	Feature	Subclause	Value/Comment	Status	Support
LP-01	Support for both wake modes	82.2.19.2.2	Variable LPI_FW may be true or false	LPI:O	Yes [] No [] N/A []
LP-02	Insertion and deletion of LPIs in groups of 8	82.2.3.6		LPI:M	Yes [] N/A []
LP-03	BIP statistics during LPI	82.2.8	BIP statistics not updated during LPI	LPI:M	Yes [] N/A []
LP-04	RAM insertion	82.2.9	Insertion of Rapid Alignment Markers meets the requirements of 82.2.9	LPI:M	Yes [] N/A []
LP-05	Transmit state diagrams	82.2.19.3	Support LPI operation in Figure 82–16	LPI:M	Yes [] N/A []
LP-06	Receive state diagrams	82.2.19.3	Support LPI operation in Figure 82–17	LPI:M	Yes [] N/A []
LP-07	LPI transmit state diagrams	82.2.19.3.1	Meets the requirements of Figure 82–18	LPI:M	Yes [] N/A []
LP-08	LPI receive state diagrams	82.2.19.3.1	Meets the requirements of Figure 82–19	LPI:M	Yes [] N/A []
LP-09	LPI transmit timing	82.2.19.3.1	Meets the requirements of Table 82–8	LPI:M	Yes [] N/A []
LP-10	LPI receive timing	82.2.19.3.1	Meets the requirements of Table 82–9	LPI:M	Yes [] N/A []